PRODUCTION INFRASTRUCTURE & SECURITY REPORT

Every protocol it takes to ship safely.

A researched blueprint of the services, protocols, and security program to take the non-custodial, AI-native, multi-chain wallet to an industry-grade launch — with deep dives on *security* and on how users are managed when the server holds *none* of their secrets.

10
Domains researched

0
User keys on any server

5
Highest-leverage moves

INTENT WALLET V3

Contents

Executive Summary

This report answers one question: **which protocols and services does Intent Wallet V3 need to become an industry-grade, secure, scalable product — and how are users and security actually managed along the way?** Ten domains were researched against current (2025–2026) best-in-class options. Every recommendation is filtered through the one fact that changes everything: **Intent Wallet is non-custodial.**

The thesis: non-custodial rewrites both the security model and the bill

Keys are generated and used **on the user's device**, sealed with `scrypt → AES-256-GCM`, and never touch a server. That single constraint has two consequences that shape this entire document:

1. **A total server breach cannot move a single user's funds.** The worst case is a *privacy or availability* incident, never fund loss. This is the biggest structural security advantage the product has — and every recommendation below preserves it rather than erodes it.
2. **Because vendors can only ever touch privacy/uptime data — never keys — commodity SaaS can be adopted aggressively** for everything *except* the deterministic cores that sit on the fund-loss boundary. That is what keeps the bill sane.

So the buy-vs-build line is mechanical: **buy anything whose breach costs only privacy or uptime; build and guard in-house anything that touches funds.** The on-device keystore, the multi-chain signing, and the deterministic "propose → verify → dispose" gate are the product's moat and are never outsourced.

How "user management" works when we hold none of their secrets

This is the part that most differs from a normal app. There is **no account row that owns the user's money** — *the keys are the user.* The server's only job is to bind a **principal** (a public address) to a **revocable session** (a token), and it must never be able to reconstruct a secret. Concretely:

- **Authentication = SIWE (Sign-In-With-Ethereum) + a session JWT** — already shipped. The user signs a one-time nonce; the server verifies the address. No password, no server-held credential. *(Hardening: move the JWT from HS256 → ES256 + JWKS rotation, and add device-bound refresh.)*
- **Unlock & recovery = passkeys / WebAuthn** — the single highest-leverage, doctrine-pure upgrade: a hardware-backed biometric key that never leaves the Secure Enclave.
- **Email/social onboarding (embedded/MPC wallets like Turnkey or Web3Auth) is an *explicitly-labelled, opt-in "assisted" lane only — never the silent default,** because every such provider reintroduces a custody question. Magic, Coinbase WaaS and Fireblocks are **disqualified** (semi/fully custodial).
- **KYC lives at the fiat on-ramp, not in the wallet.** The on-ramp provider (Ramp / MoonPay / Stripe) does the identity check; the core wallet stays KYC-free and non-custodial.

The five highest-leverage moves

#	MOVE	WHY IT MATTERS
1	**Commission the pre-GA security audit of the on-device core** (Cure53 for the app + Trail of Bits for the crypto/guards)	The single most important line item before real-fund launch. For a wallet, the audit target is *key management + signing + the deterministic gate*, not a smart contract. A public report is a **conversion asset**.
2	**Add passkeys / WebAuthn** for unlock + recovery	Removes the seed-phrase for a whole class of users, doctrine-pure, no server involvement.
3	**Wire Blockaid into the safety gate** for transaction simulation + drainer/scam detection	Turns Chapter 10's security promise into a real, layered pre-sign defense. It *informs* the deterministic verdict, never overrides it.
4	**Harden auth**: JWT HS256 → ES256 + JWKS, device-bound refresh, private→public bug bounty (Immunefi) after the audit clears	Closes the server-side replay/mint risks; earns continuous scrutiny.
5	**Enforce RPC polling discipline** (cache, batch, back off)	RPC calls — not user count — are the dominant cost driver. Discipline here keeps the bill honest at scale.

Cost, at a glance (order-of-magnitude, honest)

The three things that move the number are **RPC calls, LLM tokens, and audits** (one-time capex). Everything else is rounding error until real scale.

PHASE	SOFTWARE RUN-RATE	ONE-TIME
Phase 0 — MVP (testnet + capped mainnet, dogfooding)	**~$0–150 / mo** (free tiers; the stack is already wired)	internal review only
Phase 1 — private beta (real mainnet, capped funds)	**~$1k–3k / mo** (Helius $499 Solana-mainnet floor, cached Claude, real infra + Cloudflare)	**$50k–150k pre-GA audit**

PHASE	SOFTWARE RUN-RATE	ONE-TIME
Phase 2 — public / scale	**~$10k–50k+ / mo** (RPC dominates; HA multi-region)	recurring audits + bug-bounty pool

How to read this report

Part I — Security and **Part II — User Management & Identity** are the two deep dives you asked for, and they lead. **Part III** covers the core infrastructure (hosting, data, blockchain, AI), **Part IV** covers operations, and **Part V** ties it together into a phased roadmap, cost tiers, and a single recommended reference-stack table. Every option is judged honestly — genuine pros *and* cons — against the non-custodial doctrine, with sources cited inline.

PART I

Security

The pre-sign safety net and the security program that earns real-fund trust.

01 Transaction Safety & On-Chain Threat Intelligence — The Pre-Sign Safety Net

This is the engine behind Bible Chapter 10 and the missing half of `packages/risk`. The doctrine makes the vendor question unusually clean: **every service in this domain is a read/analysis API**. You send it public, unsigned material — an *unsigned* transaction payload, a recipient address, a token contract, a dApp URL — and it returns a verdict. **None of them ever need a private key, a seed, or a server-held secret.** They are therefore all doctrine-compatible in principle; the real discriminators become coverage (EVM + Solana + Bitcoin), whether the call *leaks user intent* to a third party (the privacy trade-off), pricing, and audit posture.

Architecturally these are **inputs to the deterministic gate, never the gate itself.** The existing `ThreatIntel` interface in `packages/risk/src/intel.ts` (`isSanctioned` / `isBlacklisted` / `isKnownScamToken` / `isMaliciousContract` / `isPhishingDomain`) is exactly the seam: vendor feeds populate it, `RiskEngine` consults it *first*, and a hit is a hard-block. AI proposes, this net + the risk engine verify, the device signs. One caveat surfaced by reading the code: that interface is **synchronous/boolean** — perfect for static blocklists distributed as signed snapshots (sanctions, scam-token registries, phishing domains), but **transaction simulation is inherently async and per-tx** and returns *asset diffs*, not a boolean. So the stack needs a second seam alongside `ThreatIntel` — a `TransactionScanner` async provider — feeding the same engine.

The four layers, best-in-class for each:

Layer 1 — Transaction simulation ("what will this actually do?")

Decode the unsigned tx into human-readable asset changes (balance deltas, approvals, ownership transfers) *before* signing. This is the single highest-leverage safety feature.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Blockaid — Transaction Scanning	Simulate **+ validate** in one call; returns asset diffs *and* a malicious/warn/benign verdict	EVM (Ethereum, Base, OP, Polygon, +) **and Solana**; powers MetaMask/Coinbase/Backpack; ML threat verdict bundled, not just raw sim	**No Bitcoin**; enterprise sales, opaque pricing; you leak the unsigned tx + address to their servers	Enterprise / custom	Read-only; SOC 2-grade enterprise vendor; data leaves device to Blockaid	**Scale pick** — best-in-class verdict+sim
Tenderly Simulation API/RPC	Best-in-class raw EVM simulation: decoded traces, gas, balance/state changes, bundled sims	Extremely accurate; battle-tested; great DX; bundle sim fits approve→swap sequencing	**EVM only** (no Solana, no BTC); *raw* sim — no built-in "malicious?" verdict; API gated behind paid tier (400 TU/simulate call)	Free plan (no API) → paid TU-metered	Read-only; no secrets; hosted	Strong EVM simulation engine, but you build the verdict layer
Alchemy `simulateAssetChanges` / `simulateExecution`	Asset-change simulation on the RPC you already pay for	**Already wired**; zero new vendor; low latency; bundle sim	EVM only for asset-change decoding; Solana limited to native `simulateTransaction` / `simulateBundle` (no decoded asset diffs); no malicious verdict	Included in Alchemy plan	Read-only; same posture as your existing RPC	**MVP pick for EVM sim** — free, already integrated
GoPlus Transaction Simulation API	Sim + risk firewall for **EVM and Solana**	Free/open tier; covers Solana; combined with their token/address risk data	Younger sim product (2025); less proven than Blockaid at wallet scale	Free tier → enterprise	Read-only; open API model	**MVP Solana sim pick**

Layer 2 — Malicious-tx / dApp / drainer / phishing detection

The "you're about to sign a `setApprovalForAll` to a drainer" layer. ~$3.1B was lost to Web3 scams in H1 2025, drainers/phishing dominant — this is not optional. **Correction to the brief:** *Blowfish was acquired by Phantom (Nov 2024), not Blockaid, and is now Phantom-internal — it is no longer available as a neutral third-party vendor.* Blockaid is the independent enterprise leader here.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Blockaid	Internet-wide dApp scanning (~15M sites/day, ~500 new malicious dApps/day) + tx validation + address/token scanning	Proactive (catches drainers *before* a user reports); EVM + Solana; the vendor MetaMask/Coinbase trust	No BTC; enterprise-only pricing/contract	Enterprise / custom	Read-only; enterprise-audited; data leaves device	**Scale pick** for drainer/dApp intel

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Scam Sniffer	Malicious-website + blacklist API, malicious-signature detection	**Widest chain net: EVM, Solana, BTC, TON, Tron**; strong phishing-domain feed; API + extension	Narrower on deep tx simulation; smaller org than Blockaid	Startup-tier API	Read-only; hosted feeds	**MVP pick** — BTC + phishing coverage cheaply
GoPlus Security	Malicious-address API, URL/phishing blacklist, token blacklist	**Free & open**; EVM + Solana; also an MCP server; easy to self-host cache	Community-grade SLA at free tier; verify latency at scale	Free → enterprise	Read-only; open API	**MVP pick** — free malicious-address/URL feed
Pocket Universe	Consumer signing-safety extension	Good consumer UX reference	**Acquired by Kerberus (Aug 2025)**; consumer product, not a wallet-embeddable API	Consumer	n/a	Reference only, not a backend dependency

Layer 3 — Address / entity screening & sanctions (OFAC/UN/EU)

Compliance-grade: is the *counterparty* sanctioned or tied to a hack/ransomware? For a **non-custodial** wallet the obligation is lighter than a VASP's (you never hold funds), but screening the *recipient* before broadcast is doctrine-aligned ("fail closed") and future-proofs any fiat on-ramp partnership.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Chainalysis free sanctions oracle + API	On-chain smart-contract oracle + free REST API that flags OFAC/UN/EU-sanctioned addresses	**Free, no contract/relationship required**; oracle is a *public chain read* → **zero user data leaves to a vendor**; deployed on most EVM chains	Sanctions-only (no broad risk scoring); EVM-oriented	**Free**	Best posture in the whole doc: oracle read reveals nothing	**MVP pick** — free, most private
TRM Labs	Wallet screening, 100+ chains, sub-second API, contextual risk	Broadest chain coverage incl. non-EVM; explains *why* risky; strong real-time	Enterprise contract; you send addresses to TRM (intent leak)	~€60–150k/yr	SOC 2; hosted; read-only	**Scale pick** for full risk (not just sanctions)
Elliptic	High-volume screening; ~300M screenings/qtr; 99.99% uptime, P99 ~1.6s	Reliability/latency leader; 100+ chains; strong for regulated partners	Enterprise pricing; data leaves device	~€80–180k/yr	SOC 2; hosted	Scale alt to TRM
Chainalysis KYT (paid)	Gold-standard investigations/monitoring	Court-grade; largest coverage; investigations if you're ever hacked	Most expensive; overkill pre-GA	~€120–250k/yr	SOC 2; hosted	Post-GA / if regulated

Layer 4 — Token & contract risk

Honeypots, hidden mint/blacklist functions, fake-LP, tax traps — the "is this token itself a trap?" check, distinct from "is the counterparty bad?"

OPTION	WHAT	PROS	CONS	PRICING	VERDICT
GoPlus Token Security API	Honeypot / mint / blacklist / tax / LP analysis	**Free**; EVM **and Solana**; the de-facto community standard (used across DEX front-ends)	Free-tier SLA; snapshot freshness	Free → enterprise	**MVP + scale pick** — nothing beats free-and-good here
Blockaid Token Scanning	Token risk within the unified platform	One vendor for tx+dApp+token; strong verdicts	Enterprise-only; bundled	Enterprise	Fold in *if* you're already on Blockaid at scale

Non-custodial & privacy posture — the honest trade-off

Every option above is **key-safe**: the wallet signs on-device and only ships *public/unsigned* artifacts for analysis. The real doctrine tension is **privacy, not custody** — sending an unsigned tx + the user's address to Blockaid/Tenderly/TRM reveals *who is about to do what*, linking the user's on-device identity to a vendor's logs. Mitigations, in order of doctrine-fit:

1. **Prefer the reads that leak nothing.** The Chainalysis on-chain sanctions oracle is a public smart-contract call — no user data reaches a vendor server. Use it as the first sanctions gate.
2. **Proxy every third-party call through** `services/api`, never client→vendor directly. The wallet talks only to your backend; you control retention, strip PII, and can swap vendors. Never put an address in a URL query string (doctrine/privacy rule).

3. **Cache static feeds locally as signed snapshots.** Sanctions lists, scam-token registries, and phishing domains are the boolean `ThreatIntel` feeds — sync them server-side, verify the signature, and serve to the client so *routine* checks never round-trip to a vendor per-tx. Only the inherently-dynamic calls (per-tx simulation, live dApp scan) go out live.
4. **Fail closed, advisory-not-authoritative.** A vendor timeout or unknown-chain response must *block or warn*, never silently pass. And a vendor "benign" never *unblocks* what the deterministic detectors flagged — the vendor is one input to `RiskEngine`, which alone decides. This preserves "deterministic code verifies."

Coverage reality check: **Bitcoin is the gap.** Blockaid, Tenderly, Alchemy asset-sim, and GoPlus are EVM/Solana. Only Scam Sniffer (phishing/drainer feeds) and TRM/Elliptic (screening) meaningfully cover BTC, and *no one* offers rich BTC "asset-change simulation" the way they do for EVM — BTC's UTXO model makes native decoding your job. For BTC, lean on (a) your own PSBT decode + output/address screening via Scam Sniffer/TRM, and (b) honest "limited simulation on Bitcoin" UX rather than faking a rich preview.

Recommendation for Intent Wallet

MVP / launch (cheap, fast, doctrine-clean — mostly free): - **Simulation: Alchemy** `simulateAssetChanges` for EVM (already wired, free) + **GoPlus** simulation for Solana; honest "limited preview" on BTC via your PSBT decode. - **Malicious/drainer/phishing: GoPlus** (free malicious-address + URL/token feeds, EVM+SOL) **+ Scam Sniffer** (adds BTC/Tron/TON phishing coverage cheaply). Both feed the boolean `ThreatIntel` seam. - **Sanctions: Chainalysis free oracle + free API** — zero cost, best privacy, satisfies `isSanctioned` today. - Wire all of it behind `services/api` as a proxy; distribute static feeds to the client as **signed snapshots**; every verdict is an *input* to `packages/risk`, which stays authoritative and fails closed.

Scale / GA (when funds and volume are real): - **Add Blockaid** as the primary **simulation + malicious-tx + dApp-scan** engine (EVM+Solana) — its bundled ML verdict and proactive internet-wide dApp scanning is the class of protection MetaMask/Coinbase ship, and it collapses three layers into one contract. Keep GoPlus/Scam Sniffer as a **cheap independent second opinion** (defense-in-depth; never trust one feed). - **Add TRM Labs** (or Elliptic) for **full counterparty risk scoring** beyond sanctions — especially before any fiat on-ramp or regulated partner. Keep the free Chainalysis oracle as the always-on first gate. - **Keep GoPlus Token Security** for token-trap detection at both stages — free and genuinely good.

Net: a **layered, multi-source** stack — free/open feeds (GoPlus, Scam Sniffer, Chainalysis oracle) at MVP, upgrading to enterprise ML (Blockaid) + entity risk (TRM) at scale — all read-only, all proxied, all advisory to a deterministic gate that alone can refuse. No key ever leaves the device; the only thing to guard is *intent leakage*, and the architecture above minimizes it.

Sources: Blockaid platform/Solana/dApp scanning, Blockaid transaction security; Tenderly simulation docs & pricing; Alchemy simulation; Chainalysis free sanctions oracle & free screening; TRM Labs; Elliptic/Chainalysis/TRM comparison; Scam Sniffer; GoPlus Security API & tx-simulation launch; Phantom acquires Blowfish; Pocket Universe → Kerberus.

02 Security Operations, Audits & Compliance

Non-custody rewrites the threat model. Because the server never holds a key or seed (Doctrine §1), a total server breach cannot directly move funds — the single biggest security advantage Intent Wallet has, and every recommendation below preserves it rather than erodes it. But the server is not irrelevant. The crown jewels split in two: (1) **the client** — the on-device keystore (scrypt + AES-256-GCM), the signing paths (EVM RLP/EIP-1559/EIP-712, BTC PSBT, SLIP-0010 Solana), the encrypted backup, and the deterministic risk/policy/guard engines that "propose→verify→dispose"; and (2) **the server as a liar or a chokepoint** — it can feed a malicious plan, a fake balance, a poisoned RPC response, or a bad `minReceived`, and it can be DoS'd or supply-chain-poisoned. So the program has an unusual shape: **the deepest, most expensive audits point at client code, not a custody backend**, while the server side is rigorous web-app hygiene plus supply-chain paranoia — because the wallet ships signed artifacts to real funds.

1 · Third-party security audits (the trust anchor before real-fund GA)

For a wallet, "smart-contract audit" is the *wrong* headline — Intent Wallet deploys little/no on-chain code. What must be audited is **key management, the signing/derivation cores, the encrypted backup, and the deterministic guards** (the pure gate between plan and wire). That points at firms strong in *applied cryptography and client-side/native app review*, not just Solidity.

OPTION	WHAT	PROS	CONS	PRICING (ROUGH)	SECURITY POSTURE	VERDICT
Trail of Bits	Deep applied-crypto + protocol audit house; fuzzing, static/dynamic analysis, threat modeling [medium/sherlock]	Gold standard for cryptographic + key-management logic; will fuzz signing/derivation; credible with institutions & insurers	Premium price; long lead times (book months ahead); heavy report you must action	Scale ($150k–$400k+ per engagement)	Reviews code only; needs no secret/key; findings under NDA	**Anchor audit** for the wallet core + guards
Cure53	German firm specializing in browser extensions, mobile apps, VPNs, password managers, crypto tools [cure53.de]	Best-in-class for *client-side* (RN app, browser context, XSS, storage); recently audited Tangem mobile SDK + Psono extensions [tangem/psono]	Smaller crypto-DeFi footprint; team-day model can feel opaque	Startup–scale ($40k–$150k, sized by team-days)	Client-side/native review; no secret needed; publishes reports	**Best fit for the app layer** (Expo/RN + web)
Zellic	Rust/Solana specialists, cross-chain infra [medium]	Strong on the Solana/SLIP-0010 path & cross-chain state; fast-moving, sharp researchers	Newer brand than ToB/NCC; weighted to on-chain programs	Startup–scale ($50k–$200k)	Code-only; no custody	Good **Solana + cross-chain** second opinion
Halborn	Full-stack security (audit + pentest + red team), wallet/key-handling reviews [medium]	One vendor for wallet review *and* infra pentest + red team; exchange/infra pedigree	Broad rather than deepest-in-class on crypto internals	Startup–scale	Code + infra; no custody	**Infra pentest + red team** partner
NCC Group / Kudelski	Large, accredited crypto & app assessment practices [nccgroup]	Enterprise credibility, HSM/crypto expertise, compliance-friendly letters	Expensive, slower, more corporate	Scale	Code/infra review; no custody	Later, for **enterprise/insurer** sign-off

(OpenZeppelin — the institutional Solidity standard — is a poor fit here: it's contract-centric and Intent Wallet ships no on-chain contracts. Only relevant if that changes.)

What to actually put on the audit scope (in priority order): the seed/keystore encryption + KDF parameters; all three signing/derivation cores against known-answer vectors (BIP-32/44/84, SLIP-0010 — you already have conformance tests, so give the auditors those); the encrypted backup + restore + wipe/reveal re-auth; the deterministic risk/policy/guard engine (can it ever *fail open*? can a crafted plan bypass EIP-55 or the mainnet cap?); the SIWE session/JWT boundary and plan-ownership binding; and the client↔RPC trust boundary (can Alchemy/Helius responses lie the UI into a signature?). Publish the reports — for a non-custodial wallet, a public Cure53/ToB report is a *conversion asset*, not just risk hygiene.

2 · Bug bounty (continuous, after the first audit clears)

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Immunefi	Largest Web3 bounty market — 45k+ researchers, 650+ programs, $110M+ paid [immunefi]	Deepest crypto-native researcher pool; PoC-required, strong triage; the credibility signal users look for	You fund the pool (criticals commonly $50k–$250k+); noise if scope is loose	Startup–scale (fund a pool + platform fee)	No secret needed; you set scope	**Primary bounty** at/after launch
Cantina	Curated bounties on Spearbit's elite network; hosts Coinbase's $5M program [sherlock]	High-signal researchers + managed triage; competition + bounty combo	Fewer, higher-end researchers; more hands-on	Scale	No custody	Scale-stage, high-signal complement
HackerOne / HackenProof	Web2-scale (H1) / hybrid Web3 (HackenProof, 200+ programs) triage [sherlock]	H1 is great for the *web/API/infra* surface + fiat payouts; managed triage	H1 less crypto-native for wallet-internal bugs	Startup ($/mo + pool)	No custody	H1 for **web/app surface**; Immunefi for wallet-crypto

Sequence: **private/invite-only bounty first** (post-audit, pre-public-launch), then public. Don't open a public Immunefi program before the anchor audit closes — you'll pay for findings a $200 linter would have caught.

3 · Server-side secrets + KMS/HSM (never user keys — only API keys & JWT signing)

The only secrets the server legitimately holds are *its own*: Alchemy/Helius/Anthropic API keys, the JWT signing key, DB/Redis creds. Never a user key. The JWT signing key is the one that most deserves an HSM/KMS boundary — if it leaks, an attacker forges sessions (and with your plan-ownership binding, still can't move funds, but can impersonate and mislead).

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
AWS Secrets Manager + KMS	Native AWS secret store + FIPS-validated KMS/CloudHSM for signing keys	IAM-scoped, rotation, audit trail; **sign JWTs inside KMS** so the private key never leaves the HSM; SOC2/FedRAMP inherited	AWS-coupled; $0.40/secret/mo + API calls adds up [guptadeepak]	Startup ($)	Secrets encrypted at rest; KMS = key never exported	**MVP + scale pick** for infra secrets & JWT key
Doppler	Developer-first secret sync across dev/CI/prod [doppler]	Frictionless DX, great for a small team, good sync; free ≤3 users, ~$8/extra user [guptadeepak]	It's a sync layer, not an HSM; another vendor with broad read access	Free–startup	SOC2; holds secrets — scope its access tightly	Optional DX layer over KMS
Infisical	Open-source secrets platform; self-hostable [infisical]	Self-host keeps secrets in *your* infra; from ~$3/identity/mo	You run it (or trust their cloud); younger	Free (self-host)–startup	SOC2; self-host = data stays yours	Good open-source alternative to Doppler
HashiCorp Vault	Dynamic short-lived creds, multi-cloud	Most powerful (dynamic DB creds, transit engine)	Heavy to operate; BSL license churn + IBM cut HCP Vault Secrets tier in 2025 [guptadeepak]	Scale	Self-managed; strong	Overkill pre-scale; revisit at scale

Recommendation: MVP = **AWS Secrets Manager + KMS**, with **JWT signed via KMS/asymmetric key** (or the DB creds via IAM auth) so no signing key sits in an env var. Add Doppler/Infisical only if the team's DX genuinely needs it. Rotate the RPC/LLM keys on a schedule; alert on unusual spend (a leaked Alchemy key = a bill, not lost funds, but still a breach signal).

4 · WAF / DDoS / bot (the server as a chokepoint)

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Cloudflare	WAF + unmetered DDoS + bot mgmt + rate limiting in front of the Fastify API [cloudflare]	Unmetered L3/4 DDoS on *every* plan incl. free; WAF (Pro $20/mo), custom rules + bot mgmt (Business $200–$250/mo); Enterprise ML bot scoring	Advanced bot mgmt is Enterprise-only add-on (20–35% of contract) [underdefense]; another proxy in the path	Free→Pro $20→Business ~$250→Ent $5k+/mo	Terminates TLS at edge — keep it a *transport* proxy; no app secrets there	**MVP: Pro/Business**; Enterprise at scale

You already added app-level rate limiting + security headers (Redis-backed) — Cloudflare is defense-in-depth in front of it, plus it shields the RPC-proxying endpoints (your Alchemy/Helius keys) from being hammered. Turn on WAF managed rules, a rate limit on the auth + plan endpoints, and bot protection on signup.

5 · Dependency & supply-chain security (highest-leverage server-side risk)

For a wallet, a poisoned npm dependency in the *signing/keystore* path is a fund-loss event — this is arguably a higher real risk than a server breach. Recent npm attacks (e.g. mass-compromise of popular packages) make this non-optional.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Socket	Behavioral analysis that blocks *malicious* packages at PR time (install scripts, network/fs access, obfuscation, typosquats) [socket]	Catches novel malware *before* install — exactly the wallet supply-chain threat; strong npm coverage; PR-blocking	No SBOM/SLSA generation; npm/PyPI/Go-focused	Free (OSS)→~$20/dev/mo [socket]	SaaS reads dep graph; no secret	**MVP must-have** — the wallet's top supply-chain defense
Snyk	Post-publish CVE monitoring + fix PRs, SAST add-on	Known-CVE coverage + remediation workflow; SCA+SAST in one	SLSA/SBOM attestation gated to Enterprise; alerts *after* disclosure	Free→$25/dev/mo [socket]	SaaS; no secret	Complement to Socket (known CVEs)
Dependabot	Native GitHub dep + security updates	Free, zero-setup, in-repo	Reactive to advisories only	Free	In GitHub	Baseline — turn on now
SLSA build provenance	Sign build artifacts via GitHub Actions `attest-build-provenance` → Sigstore/Rekor; publish npm with provenance [github/npm]	Proves *your* published artifacts came from *your* CI, not a hijacked laptop; tamper-evident public log	Requires cloud CI on hosted runners; verification discipline	Free (GitHub Actions)	No secret; public transparency log	**Adopt before GA** for anything users install

Stack: Dependabot (free, now) + **Socket** (block malware at PR) + Snyk (CVEs) + **SLSA provenance** on the published web/mobile artifacts, plus pinned lockfiles and `pnpm` with `--frozen-lockfile` in CI. Add SAST (**GitHub CodeQL**, free for the repo; **Semgrep** for custom crypto-misuse rules) and DAST (**OWASP ZAP** baseline in CI) against the API.

6 · SIEM / threat detection (audit trail → detection)

Doctrine §8 already requires every risky decision logged with inputs + reason. A SIEM turns that audit stream into *detection*: guard denials spiking, a principal hammering plan endpoints, anomalous JWT issuance, RPC-key spend anomalies.

OPTION	WHAT	PROS	CONS	PRICING	VERDICT
Datadog Cloud SIEM	Detection over log/telemetry pipeline [datadog]	Likely already your observability stack (Prometheus/metrics exist) — one pane; managed rules	Cost scales with events (~$5/M events analyzed) [datadog]	Startup–scale	**MVP if on Datadog**

OPTION	WHAT	PROS	CONS	PRICING	VERDICT
Panther	AWS-native detection-as-code over a security data lake [panther]	Detections in code (fits your ethos), scales to TB/day	AWS-centric; more setup	Scale	**Scale pick**
Wazuh	Open-source XDR/SIEM	Cheapest in dollars, self-host	Staffing eats savings [wazuh]	Free (self-host)	Only if you have the SecOps talent

MVP: ship structured audit logs to whatever you already run (Datadog/Grafana) with a handful of high-value alerts. A dedicated SIEM is a scale concern, not a launch blocker.

7 · SOC 2 & the honest regulatory read

SOC 2 Type II is about *organizational* controls (access, change mgmt, monitoring) — not proof the wallet is safe. It matters for B2B/enterprise deals and investor diligence, not for a consumer non-custodial launch. Automate it with **Secureframe** (aggressive ~$5–7k entry pricing), **Drata** (engineering-heavy startups, ~$7.5–15k/yr), or **Vanta** (broadest, ~$10k+); add $15–80k in auditor fees [secureleap/soc2auditors]. Start the *evidence collection* early (it's a Type II 3–12 month observation window) but treat the cert as a **post-launch, revenue-gated** item.

Regulatory (honest, not legal advice): because Intent Wallet **never takes custody**, it is structurally *outside* the heaviest regimes. Under **MiCA**, non-custodial wallet software providers are **not CASPs** — MiCA regulates services (custody/exchange/transfer on a user's behalf), not self-custody software, and EU policymakers have signaled self-custody stays outside the perimeter [esma/cryptodaily]. In the **US**, FinCEN's long-standing position puts money-transmitter/**MSB** obligations on custodial intermediaries who accept and transmit others' funds, not on pure non-custodial software where users hold their own keys; **VASP/Travel-Rule** duties likewise attach to custodial providers. **The catch:** the moment you add a feature that touches custody or acts on the user's behalf — an *embedded/MPC/social-login wallet* (Privy, Web3Auth, Magic), a built-in *fiat on-ramp* (which brings the on-ramp partner's KYC/MSB into your funnel), or a *swap where you route/hold value* — you can pull yourself inside the perimeter. Keep those either (a) genuinely non-custodial (client-side MPC where *the user's* shares reconstruct only on-device) or (b) partner-delegated (the licensed on-ramp does KYC, you never touch fiat). **Get a crypto-regulatory opinion before shipping any embedded-wallet or on-ramp feature** — that, not the base wallet, is where custody creep and licensing risk live. Also honor **privacy/DSAR** (you hold a principal address, sessions, maybe an email for embedded flows — GDPR/CCPA apply to that PII; you already have a compliance package with retention/DSAR — wire it).

8 · Incident response & responsible disclosure

Before real-fund GA you need three cheap, high-value artifacts: (1) a `security.txt` + `security@` inbox + a **published responsible-disclosure policy** with a safe-harbor clause (so researchers report instead of dumping 0-days); (2) a written **IR runbook** with severities, a kill-switch plan (revoke all JWTs — you have sign-out-everywhere; a server-side "pause new plans" flag; a signed in-app advisory banner), and roles/on-call; and (3) a **post-mortem/ADR** template so every incident feeds the Doctrine's auditability loop. Practice one tabletop ("a malicious RPC feeds a poisoned balance" / "a dependency is compromised") before launch.

Recommendation for Intent Wallet

- **MVP / real-fund launch (do these *before* mainnet GA, in order):** (1) **Socket + Dependabot + pinned lockfiles + SLSA provenance** on published artifacts — the wallet's #1 supply-chain risk; (2) **AWS Secrets Manager + KMS-signed JWTs** — no signing key in an env var; (3) **Cloudflare Pro/Business WAF + DDoS + rate limiting** in front of the API; (4) **one anchor audit** — **Cure53** for the Expo/web client + **Trail of Bits** (or Zellic for the Solana path) for the signing/keystore/guard cores, with the reports **published**; (5) **private bug bounty** post-audit; (6) `security.txt` + disclosure policy + IR runbook + kill-switch. CodeQL + ZAP in CI are effectively free — turn them on now.
- **Scale (post-launch, revenue-gated):** public **Immunefi** program (+ **HackerOne** for the web surface); **SOC 2 Type II** via Secureframe/Drata (start evidence collection early, cert later); a real **SIEM** (Datadog Cloud SIEM, or Panther at TB scale); **NCC Group** enterprise/insurer sign-off; annual re-audits on every crypto-core change. **Gate any embedded-wallet, MPC, or fiat on-ramp feature behind a crypto-regulatory opinion** — that is the only path that can drag a non-custodial wallet into custodial licensing.

Sources: audit firms medium, cure53.de, tangem · bounties immunefi, sherlock · secrets guptadeepak, infisical · supply chain socket, github/npm-provenance · edge cloudflare, underdefense · SIEM datadog, wazuh · SOC2 secureleap, soc2auditors · regulatory esma/mica, cryptodaily, hacken/vasp

PART II

User Management & Identity

How users are authenticated, recovered, and governed — when we hold none of their secrets.

03 Wallet Key Infrastructure & Authentication (the User-Management Core)

For a non-custodial wallet the phrase "user management" is a category error borrowed from SaaS. There is no account row that owns the user's funds; **the keys are the user**, and the server's only job is to bind a *principal* (a public address) to a *session* (a revocable token) — it must never be able to reconstruct a secret. Intent Wallet already gets the hard part right: the seed is generated on-device, sealed with `scrypt(N=2¹⁵)` → `AES-256-GCM`, and SIWE proves address control without transmitting anything private (SECURITY.md §6, API.md §4). The question this section answers is: **what production-grade infrastructure surrounds that core so a normal person — even one who only has an email address — can onboard, unlock, connect to dApps, and abstract gas, without the server ever gaining the ability to move their money.** The honest headline: our own on-device keystore is *more* non-custodial than every vendor below. Everything here is either a hardening of what we have (passkeys, ES256/JWKS) or an optional convenience layer (embedded wallets, AA) whose custody trade-off must be stated out loud.

1 · Authentication layer: SIWE + sessions, hardened

SIWE (EIP-4361) is the correct, already-shipped primitive — a one-time server nonce, signed in the browser, verified by recovering the address. Two hardening items are already flagged in our own docs and remain the right calls: migrate the session JWT from **HS256 (shared secret)** → **ES256 + JWKS rotation** so a leaked API replica can't mint tokens, and add **proof-of-possession refresh** (DPoP-style, RFC 9449) binding the refresh token to a device key so a stolen bearer can't be replayed. Nonces already live in Redis with one-time consumption; that is correct and fail-closed. SIWE covers EVM natively; for a universal identity across BTC/SOL, keep the EVM address as the canonical principal and attest the BTC/SOL pubkeys under it (signed linking), rather than running three parallel auth schemes.

2 · Passkeys / WebAuthn — the unlock & recovery upgrade

This is the single highest-leverage, doctrine-pure addition. A passkey is a FIDO2/WebAuthn credential whose **P-256 (secp256r1) private key never leaves the Secure Enclave / StrongBox** — the OS only returns a signature after biometric consent [Spark]. Two distinct uses, both non-custodial:

- **As the vault-unlock gate** (near-term): replace/augment the password that derives the scrypt key with a passkey-gated, hardware-wrapped vault key — this is exactly the "OS-keystore wrap + biometric gate" already scoped for Phase 8 (ADR-0029). No server involvement, no custody change.
- **As an on-chain signer / recovery factor** (smart-account path): passkeys can *be* a wallet signer. The friction is curve mismatch — EVM has `ecrecover` for secp256k1 but no native secp256r1. **RIP-7212** fixes this with a precompile at `0x100` that cuts P-256 verification from ~300k to ~3,450 gas, already committed by Optimism, Arbitrum, Polygon, zkSync [Alchemy]; **EIP-7951** proposes the same for mainnet. Solana shipped a native secp256r1 precompile in June 2025, making passkey signers viable there too [Helius]. This only matters if we adopt smart accounts (§4) — a passkey signing an ERC-4337 UserOp with programmable social recovery removes the seed phrase for a whole class of users, without a server ever holding key material.

3 · Embedded / MPC / TEE wallet providers — the honest custody audit

These exist to let a user "sign in with Google/email" and get a wallet with no seed phrase. **Every one of them moves key custody off the user's sole device into a provider-mediated model** — that is the trade-off, full stop. The only honest question is *how reconstructable the key is by the provider or a compromised server*. Ranked from most to least defensible for our doctrine:

OPTION	WHAT IT IS	PROS	CONS / RISKS	PRICING TIER	NON-CUSTODIAL & SECURITY POSTURE	VERDICT
Turnkey	Key-management/signing API; keys generated & used inside AWS Nitro **TEE enclaves**, never leave	Keys never exist outside the enclave; policy engine for granular auth; SOC 2 Type II; independently verifiable attestation	Not a consumer onboarding product — you build the login UX yourself; per-signature cost; enclave = a server boundary (not the user's device)	PAYG $0.10/sig (≤1k wallets); Pro $99/mo, $0.05/sig; Enterprise ~$0.0015/sig [Turnkey]	**Strongest.** Keys never reconstructed outside a tamper-proof TEE; structure org policies so *the user's* auth is required to sign → provider can't unilaterally move funds	**Best embedded option if we add one**

OPTION	WHAT IT IS	PROS	CONS / RISKS	PRICING TIER	NON-CUSTODIAL & SECURITY POSTURE	VERDICT
Web3Auth (now Consensys/MetaMask)	MPC; key shares split across Torus nodes + **the user's own device share**	True non-custodial MPC (user holds a share → no single party can sign); 30+ chains incl. BTC/SOL/Aptos; cheap	~500ms signing (slower); node-network liveness dependency; acquisition churn	Free tier; Growth ~$69/mo [web3auth]	**Strong.** User-held share means provider majority can't reconstruct; avoids VASP custody classification when user provably controls a share	Scale option for email/social onboarding
Privy (now Stripe)	EOA per user; key split into **TEE (enclave) share + auth share**, reassembled *briefly inside a Nitro enclave* to sign	Excellent DX; 75M+ wallets; TEE-based ~175ms; huge distribution via Stripe	**Key is momentarily reconstructed in full** inside the enclave on every sign — researchers flag this window; Stripe ownership raises a custody-optics question	Free <500 MAU; Core $299/mo ≤2,500 MAU; usage-based above [Privy]	**Good, not pure.** No single system holds the full key at rest, but reconstruction-on-sign is a weaker guarantee than Turnkey/Web3Auth	Acceptable convenience layer; not doctrine-pure
Dynamic (now Fireblocks)	TSS-MPC embedded wallets + polished multi-wallet login/onboarding UX	Never constructs a full key (TSS); best consumer onboarding UX; SIWE built in	Fireblocks acquisition pulls it toward a custody-to-consumer stack; MPC liveness	Free early tier → MAU-based Pro/Enterprise	**Strong on key model** (TSS, no reconstruction), but watch the Fireblocks custodial gravity	Good onboarding UX pick at scale
Magic	AWS **HSM/KMS** encrypts the user's key; **full encrypted key sits in Magic's cloud DB**	Simplest email-magic-link UX; fast	**Effectively semi-custodial** — the entire key lives encrypted under AWS KMS that Magic controls; single point of failure	~$499/mo comparable tier	**Weakest of the "non-custodial" set.** Provider can technically decrypt; fails our doctrine	**Avoid**
Coinbase WaaS / CDP	Regulated WaaS + Smart Wallet (passkey-only, Base paymaster)	US-regulated; free preview; strong passkey Smart Wallet	Custodial-leaning obligations; Base/Coinbase lock-in	Free preview	Deliberately accepts custodial duties for institutions [web3auth]	**Avoid** (custody + lock-in)
Fireblocks	Enterprise MPC custody platform	Institutional-grade, 130M+ wallets	Built for **custodial** institutions; heavyweight	Enterprise only	Custodial by design	**Avoid** (custodial)

The doctrine verdict on embedded wallets: none is as non-custodial as our existing on-device vault, and all introduce PII (email/OAuth identity) that triggers **DSAR/retention obligations** we currently avoid. If — and only if — email onboarding proves to be a growth blocker, the honest picks are **Turnkey** (keys never leave a TEE) or **Web3Auth** (user holds an MPC share), used in a mode where Intent Wallet's *server* is never a key-share holder. Present it in-app as a clearly-labeled "cloud wallet (provider-assisted)" tier, distinct from the "self-custody (this device only)" default — never silently. Purchases of these are SaaS spend, not a custody transfer, but the *architecture* choice is a custody decision and belongs in an ADR with the Principal Security Engineer's signature.

4 · Account Abstraction (ERC-4337) — gas abstraction & session keys

AA is **EVM-only** but unlocks four things our roadmap already wants, all non-custodial (the smart account is owned by the user's key/passkey): **gas sponsorship** (paymaster pays fees / accepts USDC), **session keys** (the exact "automation caps + allowlist + expiry + revocation" grant model in ADR-0028), **passkey signers** (via RIP-7212), and **programmable social recovery**. Providers pair a *bundler* (submits UserOps to the EntryPoint) with a *paymaster* (sponsors gas):

OPTION	PROS	CONS	PRICING	VERDICT
Pimlico	Most neutral/portable (permissionless.js, ERC-7579 modular accounts); high bundler volume; free on testnet	Infra-only, you assemble UX	Usage-based on bundled ops + sponsored gas [eco]	**MVP pick** — vendor-neutral, no lock-in
Alchemy (Account Kit)	We already use Alchemy for EVM RPC — one vendor, one bill; bundler+paymaster+SDK	Ties more of the stack to Alchemy	Usage-based	Strong consolidation option

OPTION	PROS	CONS	PRICING	VERDICT
ZeroDev	Best DX; Kernel account; first-class session keys & passkeys	Kernel-specific account model	Usage-based	Best if session-keys/automation is the priority
Biconomy	Modular (ERC-7579), gas tank	Ecosystem churn	Usage-based	Viable alternative

Note AA covers **only EVM** — Solana uses fee-payer relayers / Squads for the same UX, and Bitcoin has no equivalent, so gas abstraction must be presented per-chain, honestly.

5 · dApp connectivity: WalletConnect / Reown

For letting our wallet connect *out* to external dApps (and be discoverable), **Reown AppKit (formerly WalletConnect)** is the de-facto standard — 500+ wallets, one-click SIWE, relay is **non-custodial** (it transports signed payloads, never keys). Free to integrate; Pro/Enterprise MAU+RPC tiers at scale [Reown]. It is a transport, not a custody surface — safe to adopt.

Recommendation for Intent Wallet.

MVP / launch (doctrine-pure, ship now): Keep the **on-device scrypt+AES vault as the sole custody model**. (1) Harden auth: **SIWE + ES256/JWKS + PoP refresh**. (2) Add **passkey/WebAuthn as the biometric unlock gate** (Phase 8, ADR-0029) — the biggest UX win with zero custody change. (3) Adopt **Reown AppKit** for dApp connections. (4) Do **not** integrate any embedded-wallet provider yet — none beats what we have, and each adds PII/DSAR surface. Optionally pilot **Pimlico + a passkey signer** on one L2 for gasless UX, gated behind an ADR.

Scale: If email/social onboarding becomes a proven growth blocker, add a clearly-labeled **"cloud wallet" tier** on **Turnkey (TEE)** or **Web3Auth (MPC, user-held share)** — never Magic/Coinbase/Fireblocks — structured so our server is never a key-share holder, with the Security Engineer's ADR sign-off. Roll out **ERC-4337 session keys (ZeroDev or Alchemy Account Kit)** to power the automation-caps grant model on EVM, with per-chain honesty about where gas abstraction does and doesn't exist. The invariant to defend at every step: **a normal user can onboard, but no server — ours or a vendor's — can ever move their funds.**

Sources: Fireblocks embedded-wallet comparison · Turnkey non-custodial key mgmt · Privy security architecture · Privy pricing · Web3Auth WaaS comparison · RIP-7212 (Alchemy) · Solana passkeys (Helius) · ERC-4337 infra 2026 (eco) · Reown

04 User Lifecycle, Privacy & Enterprise Identity

How do you "manage" a user when you deliberately hold none of their secrets? The answer reframes the whole discipline. In a custodial SaaS the user *is* a row with a password hash; if that store leaks, accounts fall. In Intent Wallet the **user is their keys**, and those keys never touch us (SECURITY.md §2.1 ranks the seed as the only catastrophic asset, and it "never leaves the device"). What the server manages is the *non-catastrophic* residue: a **principal** (a public address, recovered from a SIWE signature), one or more **sessions** (revocable JWTs), a **device registry**, **consent/retention records**, and **notification tokens**. Every one of these is a privacy or availability concern — losing all of them cannot move a cent. User management here is therefore the discipline of managing *public metadata and revocable grants*, never credentials. This is a feature, not a gap: it collapses the blast radius of every breach class except device theft.

The lifecycle: onboard → active → recover → offboard

STAGE	WHAT HAPPENS (NON-CUSTODIAL)	WHAT THE SERVER EVER SEES
Onboard	Vault generated on-device (scrypt+AES-256-GCM); user does SIWE to prove control of an address	Public address → issues a session JWT. Never a secret.
Active	Device signs; server runs Risk+Policy gate, serves reads (balances, history), pushes notifications	Principal, session, device id, push token, activity metadata
Recover	Seed phrase restore, **or** passkey-gated re-derivation of the local vault key. There is no "reset password" — we cannot recover what we never held	Nothing. Recovery is entirely device+user.
Offboard	Sign-out-everywhere (revoke all sessions — already shipped, JWT revocation), local wipe (confirmed), and a **DSAR erasure** of server-side PII	Revocation list entry; deletion tombstone

The honesty rule (Doctrine #3) bites hardest at **Recover**: we must *never* imply a server-side account recovery, because there is none. Onboarding UX must make "your seed is the only recovery" comprehensible before a signature — this is the single biggest churn and support-load risk of the whole model, and it is worth solving with great UX rather than by quietly adding custody.

Session & device management (server holds no key)

SIWE (ERC-4361) is already the auth primitive; the correct 2026 complement is **passkeys / WebAuthn** for the *re-authentication and device-binding* layer — not for holding keys. NIST SP 800-63-4 (finalized 2025) now recognizes synced passkeys as AAL2 phishing-resistant authenticators, so a passkey gate on unlock/critical actions is both defensible and standards-blessed [NIST/webauthn.me]. Concretely:

- **Device registry**: each device registers a passkey (public key) and a device label; the server stores *public* credentials only. A new device = a new SIWE + passkey enrollment, surfaced to the user as "new device signed in."
- **Session model**: short-lived access JWT + rotating refresh, each bound to a device id, all listed in a "Devices & sessions" screen with per-session **revoke** and **revoke-all** (both already implemented). Revocation is a server-side list — the one legitimate piece of auth state we hold.
- **Vault-key protection, not vault-key custody**: use the passkey (via WebAuthn PRF extension) or the OS keystore (Secure Enclave / StrongBox, mandated Phase 8 per SECURITY.md) to *wrap the local vault key* on-device. This gives biometric unlock and phishing resistance **without any secret ever leaving the device** — the doctrinally-correct use of passkeys for a non-custodial wallet.

The custody fork: embedded / MPC wallets (analyze honestly)

The market's easy-onboarding answer is **embedded wallets** with email/social login. These are genuinely non-custodial in the narrow sense (no single party holds a full key), but they introduce a **recovery-custody dependency**: the user's ability to reach their keys now depends on the provider's TEE/MPC availability and on a Web2 auth account. That is a *different* trust model than "seed on device," and it must be labeled as such, never blended silently.

OPTION	WHAT	PROS	CONS / RISKS	PRICING	SECURITY POSTURE	VERDICT
Turnkey	Non-custodial key infra; signing inside AWS Nitro **TEEs**, keys never decrypted outside enclave; built by ex-Coinbase Custody team	Verifiable (code attestation, reproducible builds); passkey-native; SOC 2 Type II	You depend on Turnkey infra availability; keys generated in *their* enclave, not your device	Usage-based; startup-friendly	Strongest of the embedded set: TEE + attestation, no full-key reconstruction [turnkey.com/docs]	**Best fit** if we ever offer a hosted-signing tier

OPTION	WHAT	PROS	CONS / RISKS	PRICING	SECURITY POSTURE	VERDICT
Privy (Stripe, 2025)	Embedded wallets blending email/social + Web3; TEE-based signing (~175ms)	Smooth Web2→Web3 onboarding; now Stripe-backed; SOC 2	Shamir reconstruction assembles a full key at sign time; custody-flavored recovery; Stripe roadmap risk	~$299/mo for 2,500 MAU; 499-MAU free tier	Non-custodial, SOC 2, but weaker isolation model than TEE-only	Good onboarding, **not doctrine-first**
Dynamic (Fireblocks, 2025)	TSS-MPC embedded wallets + polished connect UI (EVM/Solana/Cosmos)	Never constructs a full key; strong multichain UX	Now inside Fireblocks (custody-adjacent parent); MPC coordination dependency	~$249/mo for 5,000 MAU	Non-custodial MPC, SOC 2	Strong UX; parent is a custodian
Web3Auth (MetaMask/Consensys)	Social-login key management (MPC/SSS)	Cheapest social login; broad chains	Auth-only (no smart accounts/paymaster); ~500ms MPC; you assemble the rest	~$69/mo basic	Non-custodial, but more moving parts	Budget social-login only

The honest call: none of these should replace the on-device vault, which is our promise. If we want a low-friction "try it in 30 seconds" path, the doctrine-preserving move is **passkey-encrypted on-device keys** (our own vault, WebAuthn-wrapped) — no third party in the key path at all. Only if a *hosted-signing* product line is later demanded (e.g., for programmatic/agent wallets) should we reach for **Turnkey**, precisely because its TEE-attestation model is auditable and it never holds a decrypted key. That option must ship as an explicitly-labeled, separate account type — "Intent Cloud Wallet (hosted key, recoverable)" vs. "Intent Wallet (on-device, seed-only recovery)" — so the custody trade-off is the user's informed choice, never a silent default.

Enterprise identity: teams, treasury & roles — without custody

Enterprise "user management" for a wallet is **treasury + RBAC**, and it maps cleanly onto on-chain multisig plus policy, so the server still holds no key:

- **Team treasury / roles = multisig**, not a database ACL. **Safe** (EVM) secures >$100B across 100k+ orgs and is ~10x cheaper than Fireblocks/Fordefi; roles/spend-limits/approval-thresholds live on-chain and are auditable [safe.global, ridgewayfs.com]. On Solana, the analog is **Squads**. Our deterministic Policy Engine (already built, task #42–44) enforces off-chain guardrails (per-role caps, allowlists) *before* the device signs; the multisig enforces them on-chain. Neither requires us to hold a key. **Fireblocks** is the enterprise heavyweight but is hybrid-custodial (NYDFS trust) — **disqualified** as a default; **Fordefi** is non-custodial MPC for DeFi-active desks and is a reasonable *optional* integration for institutional users who want it.
- **Org login = SSO/SAML/SCIM**, cleanly separable from wallet keys. This governs *who on a team can see/propose* (the read + proposal plane), not *who can sign* (still the device/multisig). **WorkOS** is the clear pick: ~$125/connection for SSO/SCIM, AuthKit free to 1M MAU, IT-admin self-serve, SCIM directory sync for auto-provisioning/deprovisioning [workos.com]. **Auth0** gates SSO behind $1,500+/mo enterprise tiers and quotes B2B — heavier and pricier for our shape. SCIM deprovisioning is the enterprise offboarding story: when HR removes a user, their *session and proposal rights* vanish automatically; their signing rights were never ours to grant.

CONCERN	MVP	SCALE
Team treasury	Safe (EVM) + Squads (SOL), read-only integration	+ policy-engine co-signing, Fordefi optional
Org SSO/SCIM	Defer (personal wallets only)	WorkOS (SAML, OIDC, SCIM)

Privacy & data governance (PII, consent, DSAR)

Because we minimize what we hold, we minimize what we must govern — but addresses, balances, history, and intents are still PII/deanonymization vectors (SECURITY.md asset #4). Classify and split:

- **Local-only, never synced**: seed, vault, unlocked keyring, draft intents. (By construction.)
- **Synced, E2E-encrypted (we can't read it)**: contacts, labels, watchlists, settings — encrypted with a device key so cross-device sync never exposes plaintext to the server.
- **Server-side operational PII (minimal, governed)**: principal address, sessions, device metadata, notification tokens, audit logs. Retention-bounded; our Compliance package already has retention/DSAR/consent scaffolding (tasks #60–62).

For the consent + DSAR machinery, buy don't build at scale: **Osano** (from ~$199/mo, consent + DSAR intake, US privacy-law coverage) or **Transcend** (E2E-encrypted DSAR fulfillment, direct vendor connections) are the pragmatic picks; **OneTrust** is the enterprise standard but quote-only and heavy [osano.com, onetrust.com]. Right-to-erasure must honor the Doctrine's fail-closed honesty: on-chain data is immutable and cannot be erased — the DSAR flow deletes *our* off-chain PII and says so plainly, rather than pretending to erase the chain. MVP can satisfy GDPR/CCPA with our own consent records + a manual DSAR runbook given how little PII we hold; adopt a CMP when EU/UK traffic or org customers make automation cheaper than the risk.

Notifications (transactional only)

Push via **FCM/APNs** directly is free and fine at MVP for tx-status and security alerts ("new device signed in," "high-risk intent blocked"). At scale, an orchestration layer — **Knock** (from ~$250/mo, polished in-app feed components, 10k/mo free) or **Courier** (broader provider fanout, visual templates) — de-duplicates channels and manages user notification preferences (itself a consent surface) [knock.app]. Transactional **email** goes through **Postmark** (best deliverability/transactional reputation), **Resend** (best DX), or **SES** (cheapest at volume) — marketing blasts never; every message must be user-relevant and preference-gated.

KYC lives at the on-ramp, never in the wallet

The core wallet stays **KYC-free and non-custodial**. Identity verification is legally required *only* when fiat touches crypto, and that obligation belongs to the **on-ramp provider**, who is the money-services entity — not us. Embed a hosted widget and let them own KYC/AML, fraud, licensing, and the PII: **MoonPay** (160 countries, 50-state US licensing — safest default), **Ramp** (fast/low-friction KYC, many local rails, SOC/ISO certified), **Stripe Crypto Onramp** (US + 30 countries, Stripe handles KYC/fraud), **Coinbase Onramp** [moonpay.com, ramp.network, quicknode.com]. Aggregators like **Onramper** hedge coverage. The wallet passes only the destination *address* to the widget and receives funds on-chain — we never see a passport, never hold the fiat, never become a custodian. This is the cleanest possible boundary: the one place regulation forces identity, we outsource entirely and keep our core promise intact.

Recommendation for Intent Wallet. MVP / launch: Keep keys strictly on-device; add **passkeys (WebAuthn PRF + OS keystore)** to wrap the local vault — biometric unlock, phishing resistance, *zero* server-side key. Ship **device & session management** (registry + revoke-everywhere, already built) as a first-class screen. Handle **consent + DSAR** with our own Compliance package + a manual runbook (we hold little PII). **Notifications** via direct **FCM/APNs + Postmark** for transactional/security only. **Fiat on-ramp KYC** fully outsourced to **MoonPay** (default) with **Ramp** as fallback — wallet stays KYC-free. No embedded-wallet custody fork at launch. **Scale:** Add **WorkOS** (SSO/SAML/SCIM) for org accounts, **Safe + Squads** for team treasuries co-signed by our Policy Engine, and a CMP (**Osano** or **Transcend**) when EU/org volume justifies it. Layer **Knock** for multi-channel notification + preferences. *Only if* a hosted-signing/agent-wallet product is explicitly demanded, add **Turnkey** as a clearly-labeled, separate "recoverable cloud wallet" account type — chosen for its TEE-attestation model — never as a silent default that dilutes the on-device promise.

PART III

Core Infrastructure

Where it runs, where non-secret data lives, how it reaches the chains, and the AI layer.

05 Hosting, Compute & Delivery

Intent Wallet has four things to host: the **API** (`services/api` , a Fastify modular monolith over Postgres + Redis), the **web frontend** (`apps/web` , a Vite/React static SPA), the **mobile app** (`apps/mobile` , Expo/RN), and the **background workers** the roadmap already implies (automation engine, settlement coordinator, cron/scheduled tasks, webhook delivery). The infra scaffold already targets EKS + ArgoCD + multi-region ([ARCHITECTURE §9], [arch 05]) — good as a *destination*, but wildly premature as a *starting line*.

The non-custodial reframing. Custody is not a hosting property here, and it is important to be precise so we don't cargo-cult. The doctrine says *user keys/seed never touch a server* — and with on-device signing they don't, on any host. What the server legitimately holds are its **own** operational secrets: `IW_AUTH_SECRET` (session-signing), the Postgres/Redis credentials, and the **Alchemy/Helius RPC API keys**. Those are not user custody, but they are still secrets a host's platform can see at rest and in env, so "does this host handle secrets sanely (encrypted, scoped, not logged)" is the real question — not "is it custodial."

The host that matters most is the one serving the frontend, not the API. For a wallet, the browser JS *is* the key-handling surface. Whoever serves that bundle can, in principle, ship code that exfiltrates a seed the moment it's decrypted in memory. That makes frontend-host **integrity** (immutable content-addressed deploys, strict CSP + Subresource Integrity, no host-side edge-function injection into the wallet path) a first-order security control — arguably *above* API host choice. Any "edge middleware that rewrites your HTML/JS" feature is a liability we should keep off the signing path.

Compute for the API + workers (containers vs serverless)

Fastify keeps long-lived connections (Postgres pool, Redis, potential WebSocket edge) and the workers are long-running/stateful — this is a **container** workload, not a pure-serverless one. Serverless (Lambda) fights us on cold starts (our interaction budget is <100 ms), connection pooling, and long tasks. Cloud Run / Fargate are the reasonable serverless-container middle ground.

OPTION	WHAT	PROS	CONS / RISKS	PRICING TIER	SECURITY POSTURE	VERDICT
Render	Heroku-like managed containers + managed Postgres/Redis, Docker or buildpacks	Simplest real path to prod; managed PG/Redis; **SOC 2 Type II** on Org plan, audit logs, RBAC, HIPAA-eligible workspace [render/spendbase]	Free tier cold-starts; less regional reach than AWS; managed-PG ceilings at true scale	Free → **$19+/svc**, Org plan for SOC 2/RBAC [techsy]	Strong for our size: SOC 2 II, scoped env secrets, US/EU regions	**MVP pick**
Fly.io	Fires app VMs (Firecracker) close to users; multi-region by default	Best edge/latency story; cheap always-on (~$5/mo/machine); good for a globally-thin API	**Reliability is the knock** — StatusGator logged ~57 incidents in 90 days, ~1h36m median; recurring control-plane/router events [statusgator/status.flyio]	~$5/mo min machine; usage-based [saaspricepulse]	SOC 2 available; you own more of the ops burden	Great tech, **too flaky to be the sole home for a funds-moving API today**
Railway	"Deploy anything in seconds" PaaS	Best DX; instant PG/Redis; nice for internal tools/workers	SOC 2 status not clearly published; usage billing can surprise; less compliance paper for a fintech	$5 Hobby → usage [techsy]	Weaker published compliance story than Render/AWS	Good for **preview/staging + workers**, not the compliance-facing prod API
GCP Cloud Run	Serverless containers, scale-to-zero, request-based	No node mgmt; scales to zero; Google-grade compliance; regional	Scale-to-zero cold starts hurt <100 ms; min-instances removes that but costs; long workers need Cloud Run Jobs	Pay-per-request; free monthly grant	SOC 2/ISO/PCI-capable; Secret Manager; strong IAM	Strong **scale** option if we prefer serverless-containers over k8s
AWS ECS on Fargate	Managed containers, **$0 control plane**, pay vCPU/mem	No cluster fee (vs EKS $73/mo); RDS/ElastiCache; deepest compliance & regions; less complexity than EKS	Fargate carries ~40–60% compute premium for zero node-mgmt [tech-insider]; AWS ops learning curve	Usage-based; RDS/ElastiCache extra	Gold-standard: SOC 1/2/3, PCI, KMS, Secrets Manager, per-region data residency	**Scale pick (pragmatic)**
AWS EKS	Managed Kubernetes (what `infra/` targets)	Portable, ArgoCD/GitOps, multi-region, ecosystem operators	**$0.10/cluster/hr (~$73/mo) before a single container** [tech-insider]; real platform-eng cost; over-engineered pre-scale	Control-plane fee + nodes + data	Same AWS compliance surface	**Scale pick only once we have a platform team** — the infra scaffold's true destination, not day one

Reading: **containers over serverless** for this API (connection reuse, workers, latency), and **managed-Postgres-included PaaS now, AWS later**. Don't pay the EKS/distributed-systems tax before there are users — the codebase's own [ADR-0027] says the monolith→services split is "a deploy change, not a rewrite," which is exactly the posture that lets us start on Render and graduate to Fargate/EKS without re-architecting.

Web frontend host

Static SPA (`apps/web` , one `styles.css` , no SSR). We want a global CDN, immutable deploys, and — for a wallet — the ability to lock down headers/CSP.

OPTION	WHAT	PROS	CONS / RISKS	PRICING TIER	SECURITY POSTURE	VERDICT
Cloudflare Pages	Static/JAMstack on Cloudflare's 300+ PoP edge	**Unlimited free bandwidth**; fastest global TTFB; same account as our WAF/DDoS; immutable deploys; ~$5–15/mo at 100k users vs $100–300 on Vercel [speedvitals/danubedata]	Build/DX slightly less polished than Vercel; some features assume Workers	Free (unlimited BW) → $20/mo Pro [danubedata]	CF security stack native; enforce CSP/HSTS/SRI at edge; DDoS included	**MVP + scale pick**
Vercel	Premier frontend PaaS	Best DX; superb previews/observability; great for Next.js	We're not Next.js, so we pay for DX we don't use; **bandwidth gets expensive at scale**; edge-middleware could touch the wallet path (keep off)	Hobby free (100 GB) → **$20/user Pro** (1 TB) [digitalapplied]	SOC 2; fine — but overlaps nothing with our edge/WAF	Excellent, but **redundant** given we already want Cloudflare at the edge
Netlify	JAMstack host	Simple, mature	Bandwidth cost + build-minute limits bite at scale	Free → paid tiers	SOC 2	Fine third choice; no reason to prefer over the two above

For a wallet, co-locating the frontend on **Cloudflare Pages** with our WAF/DDoS/CDN collapses one vendor *and* one class of edge-injection risk into a single, hardenable surface. Pin the CSP, enable SRI on the built assets, and treat the deploy pipeline (who can push to Pages) as a key-adjacent access-control boundary.

Mobile build + OTA delivery

`apps/mobile` is Expo/RN and (correctly) excluded from the pnpm workspace, shipping via **EAS**. There's no real alternative that matches Expo's managed native builds; the choice is EAS vs self-hosting.

- **EAS Build** — cloud iOS/Android builds (now M4-Pro workers, ~1.85× faster, no price hike [checkthat/expo]); flat fee per build. Removes the "maintain a Mac build farm" burden.
- **EAS Update (OTA)** — pushes JS/asset (non-native) fixes over-the-air: Free (1k MAU) → **$19/mo Starter (3k MAU)** → **$199/mo Production (50k MAU)** + edge bandwidth [stalliontech/expo].

Non-custodial caveat on OTA: EAS Update can change the running JS **without an app-store review**. For most apps that's a feature; for a wallet it's a *policy* decision — an OTA push can alter code that sits above the signing core. Recommendation: allow OTA for UI/copy/non-signing fixes, but treat any change to the wallet/signing path as a **store-reviewed native release**, and sign/verify update bundles. (Self-hosted OTA is possible and cheaper at scale [jmensah] but adds a security-critical service we'd have to audit — not worth it pre-scale.)

Edge, CDN, WAF, DDoS

One clear answer: **Cloudflare**, which the security model already names as Zone 1 (WAF/WS) [SECURITY §zones]. It fronts both the API and the frontend: TLS/HSTS, managed WAF rules, unmetered L3/4 DDoS, rate limiting (complementing our `@fastify/rate-limit`), and bot management.

TIER	COST	WHAT YOU GET	FIT
Free	$0	DNS, CDN, **unmetered DDoS**, universal SSL, managed WAF ruleset [spendbase]	MVP baseline
Pro	~$20–25/mo	Enhanced WAF + 20 custom rules	Early launch
Business	~$200–250/mo	**PCI DSS 4.0**, 100% uptime SLA, custom WAF, custom SSL [spendbase]	When handling real-fund volume / partners

TIER	COST	WHAT YOU GET	FIT
Enterprise	~$2k–5k+/mo	Advanced Bot Mgmt, dedicated support/SLA [spendhound]	Scale

Watch the add-on line items: Bot Management (~$30+/mo on lower tiers) and usage-based Rate Limiting on hot endpoints (e.g. the SIWE nonce route) [spendhound].

Multi-region & latency

Reality check: most of our user-perceived latency is **downstream RPC** (Alchemy/Helius) and Claude, not our own compute — the API is a thin orchestration + read-cache layer. So the highest-leverage moves are (1) an edge CDN in front (Cloudflare — done), and (2) caching read models (portfolio/insights) in Redis. **Single-region API + global edge** is right for MVP; the codebase's own scale plan (regional Postgres **read replicas**, ClickHouse analytics [ARCHITECTURE §9]) is the correct *scale* answer, not a launch requirement. Keep session/nonce state in Redis so the API stays horizontally stateless and region-portable — which the current design already does.

Recommendation for Intent Wallet

- **MVP / launch: Render** for the API + workers + managed Postgres/Redis (SOC 2 Type II, simplest safe path, managed datastores), **Cloudflare Pages** for `apps/web` (unlimited bandwidth, hardenable CSP/SRI, same vendor as the edge), **Cloudflare Free/Pro** in front of both for WAF/DDoS/TLS, and **Expo EAS** (Starter tier) for mobile builds + *policy-gated* OTA. Use Railway for preview/staging if the team likes the DX. Total realistic infra cost at launch: low hundreds of dollars/month.
- **Scale:** migrate the API to **AWS ECS on Fargate** (no EKS control-plane tax until a platform team exists) with **RDS Postgres + read replicas** and **ElastiCache Redis**, KMS/Secrets Manager for the operational secrets, multi-AZ then multi-region; graduate to **EKS + ArgoCD** (the `infra/` destination) only when service-splitting and GitOps genuinely pay for themselves. Keep **Cloudflare** as the constant edge, moving to Business (PCI DSS 4.0, uptime SLA) as real-fund volume grows.
- **Doctrine guardrails, non-negotiable:** no host ever receives a user key/seed (true on all of the above — signing is on-device); the **operational** secrets (`IW_AUTH_SECRET` , DB, RPC keys) live only in the platform's encrypted secret store, never in logs or `src/` ; the **frontend deploy pipeline and mobile OTA channel are treated as key-adjacent access-control boundaries** (integrity, signed bundles, CSP/SRI), because they ship the code that touches keys; and the wallet/signing path only changes via store-reviewed native releases, never a silent OTA.

Sources: techsy.io/railway-vs-render-vs-fly-io, render.com SOC 2, statusgator.com/flyio, status.flyio.net/history, saaspricepulse railway-vs-flyio-vs-render, tech-insider ecs-vs-eks-vs-fargate, aws.amazon.com/fargate/pricing, speedvitals cloudflare-pages-vs-vercel, danubedata static-hosting-2026, digitalapplied vercel-vs-netlify-vs-cloudflare, spendbase cloudflare-pricing, spendhound cloudflare-pricing, stalliontech expo-eas-update-pricing, checkthat.ai expo-pricing, docs.expo.dev usage-based-pricing, jmensah self-hosting-ota.

06 Data, Storage & the Non-Custodial Data Rule

The wallet's data layer stores **non-secret state only** — preferences, contacts, sessions, audit logs, server-issued plans/executions, and cached balances. Never a seed, private key, mnemonic, or plaintext credential. This is already codified in the repo's `DATABASE.md` §7 ("no secret ever lives server-side") and the shipped `plans` table + Redis nonce/rate-limit stores. So the vendor question is narrow and pleasant: because we never hold custody, a total breach of our database leaks *pseudonymous metadata*, not funds. The evaluation axis is therefore **operational** (durability, cost, ops burden, at-rest encryption, SOC2, data residency, DSAR support) — not "can this vendor be trusted with keys," because no vendor ever sees one.

One disqualifier still governs: any managed feature that would require the server to *hold* a user secret is out. As we'll see, that mostly bites one specific Supabase feature (its custodial-style Auth session model competing with our owned SIWE), not the databases themselves. The repo's own code is provider-agnostic — a plain `pg` pool with parameterized SQL and a SQLite local twin — so migrating between Postgres vendors is a connection-string change, not a rewrite. That portability is a strategic asset; don't trade it away for a proprietary bundle.

Managed Postgres (the system of record)

OPTION	WHAT	PROS	CONS / RISKS	PRICING TIER	SECURITY POSTURE	VERDICT
Neon	Serverless Postgres, compute/storage separated, copy-on-write branching. Databricks-owned (May 2025).	Pure Postgres (no lock-in); scale-to-zero; instant branches map perfectly to our expand→migrate→contract discipline and per-PR preview DBs; cheapest at low traffic.	Scale-to-zero cold starts add first-request latency (bad for a wallet's auth path — pin a min compute on paid); newer ops track record; storage-heavy audit logs get pricey over years.	Free 100 CU-hrs/project; Launch ~$0.106/CU-hr compute + $0.35/GB-mo storage, minimums removed Dec 2025 [Neon/vela.simplyblock.io]	AES-256 at rest, TLS in transit, SOC 2 Type II, region-pinnable (US/EU). No secret ever needed.	**MVP pick.**
Supabase (Postgres only)	Managed Postgres wrapped in a backend platform (Auth/Storage/Realtime/Edge Fns).	Real Postgres; generous free tier; bundled Storage + Realtime if we ever want them; good DX/dashboard.	The bundle is the trap — its Auth competes with our SIWE (see below); Team-tier SOC2/ISO is gated at **$599/mo**; compute-credit model surprises at scale.	Free ($0, 500 MB, 2 projects); Pro **$25/mo** + usage ($10 compute credit, 8 GB db, 100 GB storage); Team **$599/mo** (SOC2/ISO 27001, 14-day backups, PrivateLink); Enterprise custom (HIPAA, VPC) [supabase.com/pricing; makerkit.dev]	AES-256 at rest; SOC2/ISO only from Team up; EU/US regions; RLS built-in. Postgres itself needs no secret; **Auth does introduce a custody-adjacent question** (below).	Fine as plain pg; **skip its Auth**.
AWS RDS / Aurora PostgreSQL	Fully managed Postgres; Aurora = 6-way storage replicated across 3 AZs, up to 15 replicas, Aurora Global.	Battle-tested; PITR + continuous backup to S3; sub-30s failover; Global Database for multi-region reads; matches `DATABASE.md` 's own stated Stage C escape hatch.	Highest ops complexity; priciest; per-I/O billing surprises (use I/O-Optimized); AWS Backup does **not** support PITR for RDS *Multi-AZ cluster* topology — a real footgun.	Scale tier; instance + storage-GB-mo + I/O; reserved instances for constant load [aws.amazon.com/rds/aurora/pricing; cloudzero.com]	AES-256 (KMS) at rest, IAM auth, VPC isolation, SOC2/PCI/HIPAA, every region. Enterprise-grade.	**Scale pick.**
PlanetScale for Postgres	Postgres on dedicated "Metal" NVMe (added 2025; formerly MySQL/Vitess only).	Very high IOPS/perf; strong horizontal-scale heritage; good for write-heavy futures.	New Postgres offering (less proven than its MySQL line); no free tier (hobby removed); positioned pricey/premium — overkill for our mostly-metadata write volume. *(Positioning per current knowledge; verify tiers at contract time.)*	Startup→scale, no free tier	SOC2, at-rest encryption, no secret required.	Not now; revisit only if write volume explodes.

Read of the field. Our system of record is small and metadata-shaped (plans, executions, sessions, contacts, audit) — not a high-QPS OLTP monster. Any of these runs it. The differentiators that matter for us: (1) **zero lock-**

in so we keep the `pg` -pool portability, (2) **branching** to support disciplined migrations and preview envs, (3) **cost at low early traffic**, (4) a **clean path to multi-region** at scale. Neon wins 1–3; Aurora wins 4. That's the MVP→scale arc.

Redis (ephemeral shared state: SIWE nonces, revocation, rate-limit counters)

The repo already uses Redis for exactly the right things — reconstructable/ephemeral-by-design state, never a source of truth. So the pick is about cost curve and ops.

OPTION	WHAT	PROS	CONS / RISKS	PRICING TIER	SECURITY POSTURE	VERDICT
Upstash Redis	Serverless, per-request Redis over HTTP/TCP; global replication option.	Pay-per-command fits our low-but-spiky nonce/rate-limit load perfectly; scale-to-zero; no idle cost; `GETDEL` for atomic nonce consume (our SIWE flow) supported.	Per-command math turns expensive past ~200M cmds/mo — switch to fixed plan or provisioned then; managed multi-tenant.	Free 256 MB / 500k cmds; then **$0.20/100k commands** + $0.25/GB-mo over 1 GB, no bandwidth charge; flat "fixed" plans for high throughput [upstash.com/blog]	TLS, encryption at rest, SOC2, region-pinnable. Holds only nonces/TTL markers — no secret.	**MVP pick.**
AWS ElastiCache (Valkey/Redis)	Managed node or serverless cache.	Cheapest at constant high throughput with reserved nodes; lowest intra-VPC latency when co-located with Aurora; Valkey ~20% under Redis OSS.	Idle minimum (~$6/mo serverless Valkey, ~$91/mo Redis OSS serverless) — you pay even quiet; more ops.	Serverless: storage $0.084/GB-hr + $0.0023/M ECPUs; nodes hourly [upstash.com/blog; infratally.com]	KMS at rest, VPC-private, IAM, SOC2/PCI/HIPAA.	**Scale pick** (once on AWS + >~200M cmds/mo).

Object storage (client-encrypted backup blobs, chain archive, compliance/audit archive)

This is where the non-custodial rule is most visible: `backup_blobs` holds an `s3_key` + `content_hash` + `size_bytes` and the object is **client-encrypted before upload** — the server stores ciphertext it *cannot* decrypt. No object-storage vendor is trusted with anything, so cost/egress/immutability decide.

OPTION	WHAT	PROS	CONS / RISKS	PRICING TIER	SECURITY POSTURE	VERDICT
Cloudflare R2	S3-compatible object store, **zero egress fees**.	~$0.015/GB-mo storage + **$0 egress** — ideal for user backup blobs that get downloaded on device restore; S3 API compatible so code is portable.	Fewer storage classes; less mature lifecycle/immutability tooling than S3; smaller ecosystem.	Free 10 GB; $0.015/GB-mo, modest op fees, **no egress** [cloudflare.com; leanopstech.com]	Encryption at rest, SOC2, EU/US jurisdiction options. Stores only ciphertext.	**Backup-blob pick.**
AWS S3	The reference object store; 7 storage classes, Glacier, **Object Lock (WORM)**.	Object Lock (compliance mode) is the right primitive for the **7-year append-only audit log**; Glacier Deep Archive for cheap cold financial records; deepest tooling.	Egress $0.09/GB (first 10 TB) — punishing if used for frequently-downloaded backups.	Standard ~$0.023/GB-mo + egress; Glacier far cheaper cold [aws.amazon.com]	KMS at rest, Object Lock/WORM, VPC endpoints, every compliance cert.	**Compliance-archive pick.**

Use both: **R2 for hot, frequently-restored user backup blobs** (egress-free), **S3 Glacier + Object Lock for the immutable 7-year audit/financial archive** the compliance posture demands.

Supabase, specifically — which parts fit, which don't

The founder named Supabase, so be precise. Supabase is Postgres + Auth + Realtime + Storage + Edge Functions bundled.

- **Postgres** — fits fine. It's real Postgres behind a connection string; our `pg`-pool code runs unmodified. Good DX. Caveat: SOC2/ISO 27001 is gated behind the **$599/mo Team** tier — a wallet handling real funds needs that attestation, so budget Team, not Pro, for a serious launch. Data residency is per-region-pinned.
- **Storage** — usable for the client-encrypted backup blobs (S3-backed), but R2's zero egress beats it for restore-heavy blobs. Neutral.
- **Realtime** — genuinely nice if we later want live balance/activity push to the UI. A "maybe later," not a launch need.
- **Auth — the poor fit, and the honest custody nuance.** Supabase now supports Web3 sign-in via SIWE (EIP-4361) for Ethereum and Solana [supabase.com/docs/guides/auth/auth-web3]. Technically it is **non-custodial** — it only verifies a signed message; Supabase never holds a key. So it does *not* violate the doctrine's letter. **But it's still the wrong choice for us**, for three reasons: (1) we've already built and tested our own SIWE path in Fastify with Redis nonce consume (`GETDEL`), JWT `jti` revocation, and sign-out-everywhere — adopting Supabase Auth means *replacing a working, owned, fully-auditable auth core* with a third-party session model (GoTrue) whose revocation and principal-binding semantics we don't control; (2) it forks identity into Supabase's `auth.users` table, muddying our clean "principal = public address" model and our plan-ownership binding (`DATABASE.md` §5, doctrine law #5); (3) it's a lock-in vector precisely where auditability matters most. **Doctrine law #8 (everything auditable) + the Security Engineer's veto favor keeping our own SIWE.** Net: if we use Supabase, use it as **managed Postgres (+ optional Storage/Realtime) only — never its Auth.**

PII classification, encryption-at-rest, retention & DSAR

`DATABASE.md` §8 already defines the data classes; the vendor layer must support them:

- **At-rest encryption is table stakes** and every option above provides AES-256 by default. But recognize its limit: provider at-rest encryption defends against *stolen disks*, not against a breach of live DB credentials or a malicious operator. That's exactly why the doctrine keeps secrets off the server entirely, and why the one **highest-sensitivity metadata class — consent-gated raw intent text — must get app-layer envelope encryption (a KMS-managed data key, e.g. AWS KMS) on top of the DB's own encryption**, with a 90-day TTL. Provider encryption alone is not sufficient for that class.
- **PII is pseudonymous by design** (chain addresses, contacts, device tokens) — high value to an attacker for correlation, so treat addresses as PII for erasure even though they're public.
- **DSAR / retention** is *tractable because we're non-custodial*: "delete my data" is always about metadata, never money. Financial records (executions/steps) are **pseudonymized (identity unlinked), not erased** — a 7-year AML posture — while sessions, balances projections, and raw intent text are hard-deleted or TTL-expired. Any Postgres option supports this via cascade deletes + a pseudonymization job; none of the vendors constrain it. The 7-year immutable audit trail lands on **S3 Object Lock**, not the operational DB.

Backups & DR

- **Operational backups:** Neon gives PITR via history retention (7 days on paid); Supabase Pro daily 7-day / Team 14-day + PITR add-on; Aurora is strongest — continuous backup to S3 + PITR + cross-region snapshots (mind the Multi-AZ-cluster PITR gap noted above). For a funds-adjacent record system, require **PITR + cross-region snapshot copy**, and a **quarterly restore drill** (an untested backup is a rumor).
- **Cold/compliance archive:** separate from operational backups — the 7-year financial/audit retention lives in **S3 Glacier + Object Lock (WORM)**, immutable and independently durable, so an operational-DB compromise can't rewrite history.
- **User self-custody backups:** client-encrypted blobs in R2/S3. The honest user-facing truth remains: lose the seed *and* the client backup password and **no one, including us, can recover the funds** — that's the price of non-custody, stated plainly.

Recommendation for Intent Wallet.

- **MVP / launch: Neon** (managed Postgres — pure Postgres, zero lock-in preserving our `pg`-pool portability, branching for our migration discipline, scale-to-zero economics; pin a minimum compute so the auth path has no cold-start tax) · **Upstash Redis** (per-command pricing fits our spiky nonce/rate-limit load, atomic `GETDEL` for SIWE) · **Cloudflare R2** for client-encrypted backup blobs (zero egress on restores) · **keep our own Fastify SIWE + JWT auth — do not adopt Supabase Auth.** If the founder specifically wants Supabase for its dashboard/Storage/Realtime, use it as **Postgres-only on the Team tier** (for SOC2/ISO) and still bypass its Auth. Add **AWS KMS envelope encryption** for the consent-gated raw-intent-text class from day one.
- **Scale:** migrate the system of record to **AWS Aurora PostgreSQL Global Database** (PITR, read replicas, PgBouncer, multi-region reads — exactly the Stage C path `DATABASE.md` already mandates) · **ElastiCache (Valkey)** once co-located with Aurora and past ~200M Redis commands/mo · **S3 Glacier + Object Lock** for the immutable 7-year audit/financial archive, R2 retained for egress-heavy user backups. Require SOC2 Type II across the stack and a third-party data-security audit before real-fund GA.

Sources: [Supabase pricing/supabase.com/pricing], [makerkit.dev/blog/saas/supabase-pricing], [Neon/vela.simplyblock.io/articles/neon-serverless-postgres-pricing-2026], [Upstash/upstash.com/blog/redis-

pricing-comparison-every-major-provider-in-2026-with-numbers], [ElastiCache/upstash.com/blog/aws-elasticache-pricing-explained-2026], [Cloudflare R2/cloudflare.com/pg-cloudflare-r2-vs-aws-s3], [leanopstech.com/blog/cloudflare-r2-pricing-2026], [AWS Aurora/aws.amazon.com/rds/aurora/pricing], [Supabase Web3 Auth/supabase.com/docs/guides/auth/auth-web3].

07 Blockchain Infrastructure — RPC, Indexers, Data & Oracles

This is the read/relay path: the layer that lets Intent Wallet *see* balances, prices, gas, and history, and *broadcast* the bytes the device already signed. It is the widest external-trust surface in the whole system and the doctrine is unambiguous about how to treat it — **Zone 4 is assumed adversarial**; every RPC response, quote, token metadata blob, and price is validated, bounded, and treated as attacker-controlled ([`ARCHITECTURE.md §2.3`], [`SECURITY.md §2.2`]). The good news for this domain: **none of these vendors ever needs a key or seed.** They see public addresses, signed transactions, and your infrastructure API key — never a user secret. So the security questions here are not "will they steal funds" (structurally impossible) but three sharper ones: **(1)** does the vendor become a *deanonymization* surface (IP + address correlation), **(2)** is our provider API key exposed if we call it from the client, and **(3)** can a lying provider make a guard fail *open* (mispriced asset, stale nonce, fake "confirmed"). Everything below is scored on those, not on brochure uptime numbers.

The architecture already anticipates this: `packages/providers` is a health-scored, circuit-broken, failover framework, and `packages/chains` routes every call through a `ProviderPool` behind the `BlockchainAdapter` interface. The build-out is therefore about *choosing and layering vendors into pools*, not writing new plumbing.

EVM RPC (read + `eth_sendRawTransaction` relay)

The project already runs **Alchemy**. It should stay the primary and gain a *structurally independent* secondary — not a second Alchemy key, a different operator — so a single-vendor outage can't halt broadcasts.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Alchemy	Managed multi-chain RPC + enhanced data APIs (~50 chains)	SOC 2 Type II; `alchemy_*` enhanced APIs (Transfers, Token, Portfolio); sub-50ms "Cortex" routing; 99.995% claimed uptime; supernode dedup [alchemy.com/overviews/blockchain-node-providers]	Enhanced APIs are proprietary lock-in; CU weighting makes trace/debug pricey; sees your IP+address	Free 300M CU/mo → Growth $199 (1.5B CU) → enterprise [chainnodes.org]	API key only (no user secret); SOC 2 II; domain-allowlist keys for client use	**Keep as primary**
dRPC	Aggregator routing across 50+ operators, AI load-balancer, 95+ chains	Cheapest overage (~$6/1M req, flat 20 CU/call); huge free tier (~210M CU); built-in multi-region failover; self-host NodeCore option [drpc.org/pricing]	Latency varies (third-party upstreams); best as *secondary* not sole primary	Free ~210M CU → pay-as-you-go $6/1M req	Key only; can proxy; open-source NodeCore reduces vendor trust	**Best MVP secondary**
QuickNode	Multi-chain incumbent (80+ chains) inc. BTC + SOL under one contract	One vendor for EVM+BTC+SOL billing; add-ons (Blockbook, mempool); 99.99% SLA	Credit weighting varies wildly per method; SOL depth trails Helius	Free tier → paid ~$49+/mo, credit-metered	SOC 2; key only	**Scale: consolidation play**
Infura	Consensys-owned, MetaMask's backend; ~12 core chains + DIN	Battle-tested; DIN decentralizes upstreams; narrow but rock-solid EVM coverage	Narrowest chain coverage; 2026 credit-model change; Consensys data-policy scrutiny	Free → credit tiers	Key only; established compliance	**Optional 3rd for EVM redundancy**

Client-key nuance (a real finding): the web app can broadcast `eth_sendRawTransaction` *directly* from the browser ([`ARCHITECTURE.md §2`]). That is doctrinally fine (the signature is minted on-device), **but it ships the Alchemy key to the client.** Use **domain/bundle-allowlisted keys** (Alchemy and Infura both support this) or proxy broadcasts through `services/api`. A leaked unrestricted key is a billing/DoS problem, never a fund-loss one — but it still violates least-privilege.

Solana RPC (already on Helius)

Helius is the correct primary and should stay. Solana is unusual: transaction *landing* during congestion depends on **stake-weighted QoS**, so the RPC choice is performance-load-bearing in a way EVM's isn't.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Helius	Solana-native RPC + DAS API, webhooks, priority-fee & Enhanced-Tx endpoints	Best SOL product depth; DAS API for (compressed) NFTs saves weeks; real validator stake → SWQoS landing; most reads 1 credit [helius.dev/pricing]	Solana-only (need EVM/BTC elsewhere); some endpoints proprietary	Free tier → paid from $49/mo	Key only; sees IP+address	**Keep as primary**
Triton One	Yellowstone-gRPC originators; bare-metal, staked paths	Reference-grade streaming/landing; every customer gets staked connections; ShredStream	Enterprise pricing; dedicated nodes ~$2,900/mo; overkill pre-scale	Dedicated ~$2,900/mo+	Key only; enterprise	**Scale-only, if landing SLAs bite**
QuickNode	Multi-chain SOL support	Consolidates with EVM/BTC billing	SOL feature velocity trails Helius	Credit-metered	SOC 2; key only	**Failover secondary**

Recommendation: Helius primary, **QuickNode Solana as failover** in the same `ProviderPool` (one less vendor to onboard since it also covers BTC/EVM). Defer Triton until landing-rate SLAs during congestion justify the spend.

Bitcoin (the redundancy gap)

BTC is the thinnest leg today and the highest-leverage add. A wallet needs UTXOs, xpub-scan history, fee estimates, and `sendrawtransaction`. Bitcoin Core's own JSON-RPC does **not** expose address/xpub balances — you need an indexed layer (Esplora or Blockbook).

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
mempool.space API	Explorer API + best-in-class fee estimation	Gold-standard fee buckets (fastest/30m/1h/economy); free public tier; self-hostable	Public tier rate-limited; not an SLA product	Free; self-host for scale	Key-less public API; **self-host = zero third-party leak**	**Primary for fee estimation**
Blockstream Esplora	Open-source REST API (UTXOs, tx, mempool, fees) + new Electrum RPC	Free hosted `blockstream.info/api`; the reference Esplora impl; fully self-hostable	Read-only REST (not Core RPC shape); hosted tier best-effort	Free hosted; self-host	Key-less; open-source; **self-host for privacy**	**Primary for UTXO/history**
QuickNode BTC	Hosted BTC RPC + Blockbook (`bb_*`) add-on	xpub/UTXO/history via Blockbook; 99.99% SLA; same vendor as SOL/EVM	Credit-metered; hosted = address+IP visible to vendor	~10M credits/mo tiers	SOC 2; key only	**SLA-backed secondary/broadcast**

Recommendation: Esplora (self-host or hosted) for UTXO/history + mempool.space for fees as the free MVP, with **QuickNode BTC as the SLA'd broadcast/secondary**. Self-hosting Esplora later is the strongest *privacy* posture in the whole stack — no third party learns which addresses a user owns.

Indexers & historical data (activity feeds, portfolio history)

This powers the Activity tab and portfolio charts. Two shapes: **subgraph/GraphQL** (custom event indexing) vs **pre-indexed data APIs** (turnkey transfers/balances).

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Alchemy Transfers/Portfolio API	Turnkey cross-chain transfers, balances, token/NFT metadata	Zero indexing infra; already our vendor; one key covers RPC+data	Proprietary; not customizable; EVM-centric	Bundled in CU	Key only; SOC 2 II	**MVP pick — no new vendor**
Goldsky	Hosted subgraphs + real-time streaming/webhooks/SQL mirror	"Live-streamed" data; subgraph-compatible; far faster than The Graph hosted	Newer vendor; cost scales with usage	Free plan → usage-based	Key only	**Scale: custom real-time indexing**

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
The Graph (Network)	Decentralized GRT-based subgraph indexing	Decentralized, censorship-resistant; ecosystem standard	**Hosted service fully deprecated 2026**; must use paid decentralized network; slower on hot paths	GRT query fees	Fully decentralized; key-less	**Only if decentralization is a hard req**
Dune	SQL analytics over pre-indexed chains	Great for internal analytics/dashboards, not user-facing latency	Read-only, not customizable; analyst tool not app backend	Free → team plans	API key	**Internal analytics, not app path**

Note the churn: **The Graph's hosted service and Alchemy Subgraphs were both sunset (2025–2026)** — don't build on a hosted-subgraph assumption. For an app that mostly needs "recent transfers + balances," Alchemy's data APIs avoid an indexer entirely at MVP; graduate to **Goldsky** only when you need custom, real-time, streamed indexing.

Price feeds & oracles (fail-closed's front line)

This is where "never fake data" and "fail closed" get real: an unpriced or mispriced asset must **block**, not guess. Two fundamentally different tools — **HTTP price APIs** (fiat valuation for the portfolio UI) vs **on-chain oracles** (trust-minimized prices for any in-protocol logic).

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
CoinGecko API	Broad market-data HTTP API (prices, OHLC, categories)	Widest coverage; flat 1-credit/call (500 tokens = 1 credit); generous free tier (30 rpm)	Off-chain (trust CoinGecko); paid tiers ~$129+/mo	Free → Analyst $129/mo	API key (provider secret, server-side); no user secret	**MVP pick for fiat valuation**
DefiLlama	Free TVL/DeFi + price API	Free; strong DeFi/long-tail token coverage; good fallback source	Best-effort (no SLA); DeFi-skewed	Free (Pro tier exists)	Key-less/light key	**Free second price source (cross-check)**
CoinMarketCap	Market-data HTTP API	Cheaper entry ($79/mo); strong brand data	Scaled credit model (1 credit/100 pts); free tier non-commercial + 9 endpoints only	$79 → $875/mo	API key	**Alternative, weaker free tier**
Pyth Network	Pull oracle, first-party publishers, sub-second + confidence intervals	On-chain, first-party (exchanges/MMs); **confidence interval** is a native fail-closed signal; strong Solana + EVM	Pull model needs update tx; smaller TVS share (~6%)	Protocol/gas cost	Fully on-chain; **no secret**; trust-minimized	**Best for any in-protocol price logic**
Chainlink	Push oracle, 68.9% oracle market share; Data Streams for low-latency	Most battle-tested; conservative risk controls; deep EVM	Push model = update cadence; heavier EVM-centric	Protocol/gas cost	Fully on-chain; **no secret**	**Conservative on-chain fallback**

Design rule: use **CoinGecko (primary) cross-checked against DefiLlama** for the *display* valuation in `packages/portfolio`, and treat a divergence beyond a bound as "unpriced" → the asset shows honest "price unavailable," never a fabricated number. Reserve **Pyth/Chainlink** for any logic that gates money on price (slippage floors, mainnet-cap valuation) — there, an on-chain oracle with a **confidence interval** (Pyth) is exactly the fail-closed primitive the doctrine wants: widen-confidence or stale → block.

Gas estimation

EVM: prefer **EIP-1559** `eth_feeHistory` from the primary RPC plus Alchemy's gas-price endpoints; keep the pure `packages/gas` engine as the bounded *decide-not-act* layer (it already caps params). BTC: **mempool.space fee buckets** are the reference. SOL: **Helius priority-fee endpoint**. All of these are advisory inputs to a *deterministic, capped* estimator — never trust a provider's number unbounded (a lying feed that returns a huge fee is a griefing vector; the cap fails closed).

Reliability & the privacy finding

Redundancy is already structural (`packages/providers` health-scores and circuit-breaks). The build-out is populating each chain's pool with **operator-diverse** vendors (Alchemy+dRPC for EVM; Helius+QuickNode for SOL; Esplora/mempool+QuickNode for BTC) so no single operator outage stops reads *or* broadcasts, and rate-limit exhaustion on one key sheds to another.

The one genuinely underweighted risk in this domain is **privacy, not custody**: every hosted RPC learns *IP ↔ address ↔ balance ↔ intent-timing* correlations. That is Doctrine asset #4 (user privacy). Mitigations, in order of strength: (1) **proxy all client RPC through** `services/api` so the vendor sees your server, not the user's IP (also hides API keys); (2) rotate/allowlist keys; (3) offer **self-hosted Esplora / self-hosted dRPC NodeCore** for privacy-sensitive users at scale. This is the correct place to spend the "special depth on security" budget for this layer.

Recommendation for Intent Wallet

- **MVP / launch:** EVM = **Alchemy (primary) + dRPC (secondary)**; Solana = **Helius (primary) + QuickNode (failover)**; Bitcoin = **Blockstream Esplora + mempool.space fees (primary) + QuickNode BTC (SLA secondary)**; data = **Alchemy Transfers/Portfolio APIs** (no separate indexer); prices = **CoinGecko cross-checked with DefiLlama**; on-chain price logic = **Pyth**. Every hosted RPC key domain-allowlisted or proxied via `services/api` . All slot into existing `ProviderPool` s.
- **Scale:** add **Triton** (Solana landing SLAs), **Goldsky** (custom real-time indexing), **self-hosted Esplora + dRPC NodeCore** (privacy + cost), **Chainlink** as a second on-chain oracle, and consider **QuickNode** as a consolidation vendor across all three chains. Add **Pyth confidence-interval gating** to the money-path guards. Prioritize the **RPC-proxy privacy layer** — it is the single highest-value security investment in this domain.

Sources: [chainnodes.org], [dwellir.com], [drpc.org/pricing], [helius.dev/pricing], [alchemy.com/overviews], [quicknode.com/chains/btc], [blockstream.com Esplora], [mempool.space/docs/api], [chainstack.com indexing], [coingecko.com/learn], [messari.io Chainlink-vs-Pyth], [docs.pyth.network].

08 AI / LLM Infrastructure — The Intelligence Layer

Intent Wallet already has the hard part right: the LLM is a *brilliant untrusted intern* caged behind a schema-forced boundary with **zero signing authority** (AI.md §1–3). That architecture — deterministic fast-path first, then a forced-tool Claude call whose `unknown` output is Zod-validated and re-checked by Risk + Policy + a device signature — is the moat. Nothing in this stack may weaken it. The job here is to pick the *production* pieces that make that cage observable, cheap, evaluable, and hardened, without ever putting a secret on a server. A useful reframe for the whole domain: because the wallet never lets a model *act*, most "LLM security products" are, for us, **telemetry and defense-in-depth — not the load-bearing gate**. That distinction drives every verdict below.

1. The LLM provider — Anthropic Claude (keep it)

Claude is already wired and is the right long-term call: it leads on instruction-following and structured/tool use, and — critically for non-custodial — offers **Zero Data Retention (ZDR)** by agreement, **no training on API data** by default commercial terms, **SOC 2 Type II**, and a **HIPAA BAA** on the Enterprise/direct-API tier [Anthropic data retention]. The API never needs a *user* secret — only our server-side `IW_LLM_API_KEY`, which is itself optional (the wallet degrades to the deterministic path with no key). That is a clean non-custodial fit: the model sees redacted symbols and contact names, never keys or full addresses (AI.md §7).

Current pricing and the model-routing implication (per 1M tokens) [Claude pricing; TLDL]:

MODEL	INPUT	OUTPUT	ROLE IN INTENT WALLET
Haiku 4.5	$1	$5	Classification, disambiguation, the *classify* default (`IW_LLM_MODEL_CLASSIFY`)
Sonnet 5 / 4.6	$2–3	$10–15	Intent parsing fallback (`IW_LLM_MODEL_PARSE`), Copilot prose
Opus 4.8	$5	$25	Rarely — only genuinely hard multi-step planning; not the hot path

Sonnet 5 carries **intro pricing of $2/$10 through Aug 31 2026** [TLDL]. Cost levers stack: **prompt caching (–90% on cached input)** for the fixed system prompt + tool schema, and **batch (–50%)** for offline eval/replay runs. **Latency** is the reason Claude sits on a *fallback* path only — the deterministic `CompositeParser` answers most utterances in <5ms, so the sub-100ms interaction budget never depends on a network round-trip. Keep that ordering; it is both a cost and a UX win.

Pros: best-in-class structured/tool use; ZDR + SOC 2 + BAA; no-training default; already integrated behind an injectable `LlmClient`. **Cons:** single-vendor dependency; no ZDR unless you *request* it (do so before mainnet GA); output tokens are the expensive side, so keep Copilot prose terse. **Non-custodial posture:** excellent — no user secret ever transits; server key is gitignored env, leak-scanned per commit (CLAUDE.md §8).

Second-source note: keep the `LlmClient` seam vendor-neutral so a future Bedrock-hosted Claude or an OpenAI/Gemini fallback is a config swap, not a rewrite. Do **not** adopt a multi-provider router that requires shipping prompts through a third party (see §5).

2. The schema-forced boundary — adopt native Structured Outputs

Today the boundary is *forced tool* + `IntentSchema.safeParse` — belt-and-suspenders that works. Anthropic shipped **Structured Outputs** (public beta, header `structured-outputs-2025-11-13`) which **compiles the JSON schema into a grammar and constrains token generation at inference** — the model literally cannot emit off-schema tokens, giving `strict: true` tool use [Structured outputs; HN]. This *strengthens* our cage: fewer retries (lower cost + latency), and the forced-tool path becomes provably on-shape before Zod even runs.

Recommendation: enable `strict: true` on the `emit_intent` tool and keep Zod as the trust boundary. This is doctrine-compatible: structured outputs make the *shape* guaranteed; Zod + the downstream gate still own *trust*. Never treat "it validated" as "it's safe" — a well-formed intent to a sanctioned address is still `block`. Guard against beta churn (pin the header; keep the current forced-tool path as fallback if the beta returns an error).

3. Prompt-injection & jailbreak defense

Prompt injection is **OWASP LLM01 — still #1 in 2025** [Introl]. Our honest position: the wallet's *primary* defense is structural and already built — untrusted text rides in a `user` message (never the system prompt), the model has **no fund-moving tool to reach for**, a deterministic `looksLikeInjection` veto forces suspicious fund-moving intents to `clarify`, and Risk + Policy + the device signature sit downstream of *all* parsing (AI.md §8). A jailbroken model can at worst emit a weird intent that the gate refuses. So external detectors are a **monitoring and defense-in-depth** layer, not a gate we depend on.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
In-house injection veto	`looksLikeInjection` + schema gate + downstream Risk/Policy	Deterministic, offline, tested (golden corpus red-team), zero data egress	Pattern-based; not a semantic classifier	Free	Best — nothing leaves the box	**Keep as primary**
Lakera Guard	Hosted injection/PII classifier, 98%+ detection, <50ms [appsecsanta]	Strong detection, low latency, mature	**Sends prompt text to a third party**; acquired by Check Point 9/2025 (roadmap risk)	Startup→scale (usage)	Data egress unless self-hosted tier; SOC 2	Optional signal, self-host or skip
Meta Prompt Guard 2	Open 22M/86M classifier	Free, self-hostable, no egress	**100% evasion demonstrated** vs it and Azure Prompt Shield [Introl]	Free (OSS)	Runs in-house	Weak as a gate; fine as a flag
LLM Guard (OSS)	Regex/severity input+output scanners	Free, local, composable	Maintenance burden; regex ceiling	Free	In-house	Mine ideas for our veto
Rebuff	Self-hardening detector + canary tokens	Clever design	**Project archived May 2025**	—	—	Do not adopt

Recommendation: do **not** put a hosted classifier inline on the signing path — it adds latency, a data-egress surface, and a false sense that detection is the defense. If you want a second opinion, run **Meta Prompt Guard 2 self-hosted** as an *async telemetry* signal that increments a risk/observability counter (never blocks on its own; never overrides a `block`). The load-bearing defense stays deterministic and on-device-adjacent.

4. Evaluation & guardrails — verification as code

AI.md §9 already treats guardrails as executable: a **200+ utterance golden corpus** run offline asserts ≥95% parse accuracy *and* that no adversarial input yields a confident fund move; `ScriptedLlmClient` makes the orchestrator hash-stable without a live model. That is the right philosophy — extend it, don't replace it.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Promptfoo (OSS)	CLI eval + red-team, runs in CI	Free, local, no data egress, injection red-team packs, YAML-native	Not a live-trace tool	Free (OSS); paid enterprise	Fully in-house	**Adopt for CI evals + red-team**
Braintrust	Eval-first: datasets, scorers, CI gates to block prompt regressions [Braintrust]	Great regression gating, strong UI	Hosted/closed core; sends eval data out; overkill pre-scale	Startup→scale	SOC 2; data egress	Scale option if eval volume grows
NeMo Guardrails	Programmable input/output/dialog rails (Colang)	Powerful, OSS (Apache-2)	Heavy; overlaps our deterministic gate; adds a moving part	Free	In-house	Skip — we gate in code
Guardrails AI	Output validators (Py/JS)	Composable validators	Redundant with Zod + FactLedger	Free/OSS	In-house	Skip — we already do this

Recommendation: the golden corpus + `ScriptedLlmClient` are your eval core; add **Promptfoo** in CI to formalize adversarial red-teaming and to gate prompt/model changes (block a merge that drops parse accuracy or lets an injection through). Skip the guardrail *frameworks* — NeMo/Guardrails-AI solve "the model can act, constrain it," a problem we designed away. Our FactLedger + `verifyNarrative` + confidence floor already do output validation with tested adversarial cases.

5. Observability for LLM calls

You must see every model call — tokens, cost, latency, retries, the injection veto rate, which utterances fell through to the LLM vs the deterministic path. The **critical non-custodial constraint**: an observability tool logs *prompts and responses*. Even though we redact keys/addresses (AI.md §7), utterances can contain PII, so this is a **data-residency and DSAR** decision, not just a dashboard choice. Prefer a tool you can **self-host** so prompt text stays in your infra.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Langfuse	OSS span-tracing, OTel GenAI, prompt mgmt, evals [Langfuse]	**Self-hostable free (MIT)**, SOC 2 Type II, ISO 27001, GDPR, HIPAA-ready region, data residency control	v3 self-host needs a ClickHouse cluster (~$200–800/mo infra)	Cloud: free Hobby (50k units) → Core $29 → Pro $199; self-host free	**Best** — self-host keeps prompts in-house	**MVP + scale pick**
Helicone	Proxy/gateway that logs at the wire [Helicone]	Dead-simple, free 100k req, caching/rate-limit built in, self-hostable (Docker/k8s)	**Inline proxy sits on the request path** — a new availability + data-custody dependency in front of signing-adjacent calls	Free → Pro $25 → Enterprise (SOC 2)	Proxy sees all traffic; self-host mitigates	Avoid inline; async only
Braintrust	Trace logging + eval [Braintrust]	Excellent eval/regression story	Hosted-first; data egress; pricier	Startup→scale	SOC 2; egress	Overlaps §4; scale-only

Recommendation: Langfuse, wired via **async OTel spans** from `services/api` (never as an inline proxy — do not let observability become an availability dependency in front of a signing-adjacent call). Start on **Langfuse Cloud free/Core** for the MVP; **self-host** (Langfuse + ClickHouse) before mainnet GA so no user utterance leaves your infrastructure. Explicitly avoid Helicone's *proxy* mode: routing all LLM traffic through a third-party gateway contradicts the "fail closed, minimize egress" instinct even if convenient.

6. Cost control

Costs are already structurally low because the deterministic parser handles most traffic and the LLM is a fallback (AI.md §10). Stack the rest: (1) **prompt caching** on the static system prompt + `emit_intent` schema (–90% cached input); (2) **model routing** — Haiku for classify, Sonnet for parse, Opus almost never; (3) `max_tokens` **caps + terse prose** (output is 5× input cost); (4) **batch (–50%)** for eval/replay; (5) a **per-principal spend cap** in `services/api` so a hostile actor can't run up the bill (fail closed to the deterministic path on breach — honest degradation, never a fake answer); (6) track cost-per-intent in Langfuse against the deterministic-vs-LLM hit ratio as a first-class SLO.

Recommendation for Intent Wallet

- **MVP / launch:** Keep **Anthropic Claude** (Haiku-classify / Sonnet-parse routing, prompt caching on the fixed prompt+schema). Enable native **Structured Outputs** `strict:true` on `emit_intent`, Zod still the trust boundary. Injection defense stays the **in-house deterministic veto + schema gate + downstream Risk/Policy/signature** — no hosted classifier inline. Evals: the **golden corpus +** `ScriptedLlmClient`, plus **Promptfoo** red-team in CI. Observability: **Langfuse Cloud (free/Core)** via async OTel. Request an **Anthropic ZDR agreement** before touching mainnet.
- **Scale: Self-host Langfuse** (ClickHouse) so no utterance leaves your infra; add **Braintrust** only if eval/regression volume justifies it; run **Meta Prompt Guard 2 self-hosted** as an *async* injection-telemetry signal (never a blocker). Keep the `LlmClient` seam vendor-neutral for a Bedrock-Claude or fallback provider. Enforce per-principal LLM spend caps that fail closed to the deterministic path.
- **Non-negotiable across both:** the model never signs, never holds a secret, never has the last word on money; every prompt path is redactable, self-hostable, and DSAR-able; a model/network failure degrades to an honest `clarify`, never a guess.

IV

PART IV

Operations & Observability

Keeping it up, honest, and supported.

09 Observability, Reliability & Support Ops

Intent Wallet is non-custodial, so the observability blast radius is not fund loss — the seed never leaves the device, and no vendor here can be disqualified for "holding a key," because none of them ever touch one. The catch is the mirror image: **telemetry itself becomes the sensitive surface.** Errors, traces, session replays, analytics events and support chats can leak the things the server is *supposed* to know only in the aggregate — wallet addresses (pseudonymous but permanently linkable), IPs, balances, and free-text intent strings that may contain amounts, addresses or personal notes. So the doctrine ("never fake data," "everything auditable," "fail closed," no surveillance) applies to our *own* dashboards. Two hard rules govern every choice below:

1. **Never instrument `packages/core`, the keystore, or any signing path with a tool that can serialize state.** No breadcrumb, error payload, or replay may ever carry seed/private-key material. Error messages are scrubbed of secrets before they leave the process.
2. **Treat all telemetry as Zone-3/4 PII.** Scrub server-side *and* client-side, keep raw personal data out of third-party clouds where possible, and honor retention/DSAR (see `DATABASE.md`).

The project already exposes Prometheus `/metrics` (RED signals + Node process metrics) and propagates trace context — so this is about choosing the backends and the instrumentation seam, not starting from zero.

Error tracking

OPTION	WHAT	PROS	CONS / RISKS	PRICING	SECURITY POSTURE	VERDICT
Sentry	Error + performance + replay, JS/RN/Node SDKs	Best-in-class RN/Expo + Node support; server-side + client PII scrubbing; replay "private by default" (all text/inputs redacted before leaving browser); EU data region; SOC 2	Session Replay is a live footgun on wallet screens; event-based bill scales with volume; self-host is heavy	Free 5k errors/1 user; Team ~$26/mo (50k errors, annual); Business ~$80/mo [sentry.io/pricing]	SOC 2; ingestion-time scrubbing before disk; EU residency; can run a local Relay to scrub before egress [docs.sentry.io]	**Pick (MVP + scale)**
GlitchTip	Open-source, Sentry-SDK-compatible	Self-hosted → PII never leaves your infra; drop-in for Sentry SDKs; cheap	Fewer features (no rich replay/perf); you run it	Free (self-host) / small hosted tier	Fully self-hosted = strongest data-residency story	Scale fallback if data-residency demands it
Bugsnag / Rollbar	Error tracking	Mature, stable	No real edge over Sentry for our stack; RN parity weaker	Similar tiers	SOC 2	Skip

Non-custodial notes that are load-bearing: disable Session Replay by default; if enabled at all, force `maskAllText` / `maskAllInputs` / `blockAllMedia` and add an explicit `block` on any node that renders a balance, address, or the seed-backup/reveal screen (better: never mount replay on those routes). Use `beforeSend` to drop `req.body`, headers, addresses and intent text; enable Sentry's Advanced Data Scrubbing for defense-in-depth. Sample traces (e.g. 10–20%) to control cost and exposure. This gets us honest crash visibility on web + Expo without turning Sentry into a surveillance store.

Metrics, logs, traces & dashboards

The instrumentation layer should be **OpenTelemetry (OTLP)**, not a vendor SDK. OTel is now the default cross-signal standard (logs went GA in 2025) and is the litmus test for avoiding lock-in: instrument once, ship OTLP from an edge Collector, and swap backends with a config change instead of a rewrite [opentelemetry.io; grafana.com/blog]. An OTel Collector also becomes a natural **PII-scrubbing choke point** (redact/hash attributes before export).

OPTION	WHAT	PROS	CONS / RISKS	PRICING	SECURITY POSTURE	VERDICT
Grafana Cloud	Managed Prometheus/Loki/Tempo + Grafana	Generous free tier (10k series, 50 GB logs, 50 GB traces, 14-day); native to our Prometheus `/metrics`; ~$18/host vs Datadog $31–40; OTLP-native	Query/UX rougher than Datadog; alert tuning is on you	Free forever; Pro from ~$19/mo [grafana.com/compare]	SOC 2; EU hosting; data is metrics/logs, not keys	**MVP pick**
Self-host LGTM	Prometheus + Loki + Tempo + Grafana on our infra	Zero telemetry leaves our VPC; no per-GB bill; full control of retention	Real ops burden (storage, scaling, upgrades)	Infra cost only	Strongest residency/privacy	**Scale option** when volume/compliance justifies it
Datadog	All-in-one metrics/logs/APM/RUM	Polished, powerful correlation, strong synthetics	Expensive and unpredictable at scale (~$21.6k/mo @ 200 hosts/1 TB vs Grafana ~$17k); easy to over-ingest PII via RUM	Free tier thin (5 hosts, 1-day retention); per-host + per-GB	SOC 2; but RUM/logs invite PII sprawl	Only if a team already lives in it

For logs: structured JSON with a request/trace id, secrets and addresses redacted at the logger (the repo already has a `packages/observability` logger seam). Never log intent text raw.

Uptime, synthetics & status page

Synthetics must be **honest probes, not fabricated green.** Health checks should hit the real `/readyz` (which already gates on real dependencies) and run a read-only user journey (load home, fetch a testnet balance) — never a synthetic that broadcasts a transaction or touches a seed.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Better Stack	Uptime + synthetics + on-call + status page in one	All-in-one; real-Chrome transaction checks; screenshot-on-failure; free tier (10 monitors) + hosted status page	Per-responder pricing adds up; jack-of-all-trades	Free; paid from ~$29/mo (annual) [betterstack.com]	SOC 2; probes external, no secrets	**MVP pick** (covers 3 needs at once)
Checkly	Monitoring-as-code, Playwright synthetics	Best deep multi-step browser flows; versioned in git	Not a status-page/on-call product	Free Hobby; Team ~$40/mo [betterstack]	SOC 2	Add at scale for rich flows
UptimeRobot	Simple uptime	Cheap, free 50 monitors	No real synthetics/journeys	Free; Team ~$38/mo	Basic	Fallback only
Instatus	Dedicated status page	Flat pricing, no per-seat, 21 languages	Just a status page	Free; Pro ~$20/mo (vs Statuspage $29→$399→$1499) [instatus.com]	Public by design	Use if status page is split out

On-call & incident management

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
incident.io	Slack-native incident response + on-call	Modern, fast to run, IR + on-call in one; great for small teams	Newer; per-seat	~$25–31/user/mo (IR + on-call) [incident.io]	SOC 2	**Scale pick** when a real rota exists

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
PagerDuty	Industry-standard alerting/on-call	Deepest routing/escalation, huge integration set	Priciest; AIOps is a separate ~$699/mo SKU	$21 (Pro) / $49 (Business) per user/mo	SOC 2	Enterprise/large SRE only
Opsgenie	Atlassian on-call	Cheap historically	**Sunsetting April 2027 — do not adopt**	n/a	—	Avoid
Better Stack on-call	Bundled with monitoring	One vendor, one bill	Lighter IR workflow	Included in plan	SOC 2	**MVP** (bundled)

MVP reality: a two-person team does not need PagerDuty. Better Stack's bundled on-call (or a free-tier Grafana OnCall) covers a single rota; graduate to incident.io once there's a team and a real incident cadence.

Privacy-preserving product analytics

This is where the doctrine bites hardest. A wallet must **not** run surveillance analytics — no ad-tech pixels, no selling behavioral data, no fingerprinting. Wallet addresses used as analytics IDs are effectively permanent deanonymizers.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Plausible	Cookieless, aggregate web analytics	No cookies, no PII, no consent banner needed; open-source, self-hostable on a ~$10 VPS; tiny script; aligns with doctrine by construction	Aggregate only — no per-user funnels/replays	~$69/mo (1M pageviews) or self-host [posthog.com/blog]	Cookieless, no personal data; GDPR by default; self-host = data stays home	**Pick for marketing site + basic product**
PostHog	Full product analytics + funnels + flags	Powerful; can be configured cookieless, IP-anonymized, EU cloud, autocapture off; free 1M events/mo	**Collects PII by default** (user IDs, IPs, session recordings) — must be hard-locked; recordings are a wallet-screen hazard	Free 1M events/mo PAYG [posthog.com/pricing]	Configurable to GDPR; EU region; self-host	**Only if** funnels are needed *and* locked down (no recordings on wallet screens, cookieless, no address-as-id)

Rule: instrument *product events with no PII* (e.g. `intent_parsed`, `plan_confirmed`, chain used, success/fail) — never the amount, address, or intent text. Feature flags (PostHog/self-host) are legitimately useful for staged rollout of risky flows.

Customer support & docs

Support surfaces are third-party JS injected into our app — a CSP and DOM-access risk. **Never mount a support widget on the signing/seed/balance screens**, and put the anti-phishing rule in the product: *support will never ask for your seed phrase.* Route support through a help center or a screen that renders no secrets.

OPTION	WHAT	PROS	CONS	PRICING	SECURITY POSTURE	VERDICT
Crisp	Live chat + inbox + help center	Flat per-workspace pricing, generous free tier; cheap for small teams	Less depth than Intercom	Free; ~$45/mo flat [crisp.chat]	Widget = third-party JS (CSP-gate it)	**MVP pick**
Plain	API-first, Slack-native support for technical users	Fits crypto-savvy users; AI on all plans; no bloat	Newer; less "help center" polish	$29/seat + $0.99/AI resolution [plain.com]	Modern; scoped	Strong alt for technical userbase
Intercom	PLG messaging + Fin AI agent	In-app messaging, strong AI deflection	Expensive; per-resolution Fin fees	~$29–139/seat + Fin usage	SOC 2	Scale, PLG-heavy
Zendesk	Enterprise helpdesk	Ticketing/compliance depth	Heavy, pricey for a startup	From ~$115/agent	SOC 2	Only if enterprise-facing

Docs: keep the deep references (`docs/` , help center) as static, versioned, self-hosted content — cheaper, private, and doctrine-honest.

Recommendation for Intent Wallet

MVP / launch (cheap, honest, low-lock-in): - **Errors:** Sentry (free→Team) on web + Expo, Session Replay **off** on wallet screens, `beforeSend` scrubbing, never on `packages/core` . - **Metrics/logs/traces:** instrument via **OpenTelemetry Collector** (also the PII-scrub choke point) → **Grafana Cloud free tier**, feeding off the existing Prometheus `/metrics` . - **Uptime + synthetics + status page + on-call: Better Stack** free/starter — one vendor covers four needs, probing the real `/readyz` . - **Analytics: Plausible** (cookieless, aggregate) for site + basic product — no surveillance, no consent banner, doctrine-clean. - **Support: Crisp** free/flat, widget CSP-gated and off signing screens; static self-hosted docs.

Roughly $0–150/mo all-in — appropriate for pre-scale.

Scale: keep the OTel edge (no re-instrumentation) and move backends to **self-hosted Grafana LGTM** (or Datadog only if a team demands it) for residency and cost control; **GlitchTip or self-host Sentry** if data-residency requires errors to stay in-VPC; **incident.io** once a real on-call rota exists; **PostHog (EU, locked-down, no recordings)** if funnels/flags become necessary; **Intercom or Plain** as support volume grows. The through-line: OpenTelemetry keeps us vendor-neutral, everything is scrubbed at the edge, and no tool ever sees a key.

Sources: [sentry.io/pricing], [docs.sentry.io/security-legal-pii], [grafana.com/compare/grafana-vs-datadog], [opentelemetry.io], [betterstack.com/uptime], [instatus.com/vs/better-stack], [incident.io/blog], [posthog.com/pricing], [posthog.com/blog], [crisp.chat], [plain.com/blog]. Pricing verified July 2026; tiers change — confirm at purchase.

PART V

Cost, Sequencing & the Recommended Stack

What to adopt when, what it costs, and the single reference stack.

10 Cost, Sequencing & the Recommended Reference Stack

This is the synthesis layer: the other analysts prove out each domain; this section decides *what to adopt when*, *what it costs*, and *what we buy versus build*. The organizing fact is the doctrine — Intent Wallet is non-custodial, so **almost every service here is a read/relay/oracle convenience, never a custodian**. That single constraint is what keeps the bill sane: because a breached vendor can only leak privacy or availability data (never move funds), we can adopt commodity SaaS aggressively for everything *except* the deterministic cores that hold the trust boundary. Prices below are 2025–2026 list rates, verified where possible, and stated as honest order-of-magnitude ranges — real bills swing with polling frequency, chat volume, and MAU.

The three cost drivers that actually matter

Ignore the $20 line items; three things move the number: **RPC calls, LLM tokens, and audits (one-time capex).** Everything else — Postgres, Redis, error tracking, WAF — is rounding error until real scale.

- **RPC** is metered per compute unit / credit and scales with *how often the app polls balances and prices*, not with user count. Alchemy is now pure pay-as-you-go: $0.45 per 1M compute units for the first 300M, $0.40 after, with 30M CU/month free [Alchemy Pricing / alchemy.com/pricing]. Helius has a hard gotcha: **Solana mainnet requires the Business plan at $499/month** (Free and Developer/$49 are devnet-oriented), rising to $999 Professional and ~$2,900+ for dedicated nodes [Helius Docs / helius.dev/pricing]. Client-side polling discipline (cache, batch, back off) is the single biggest lever on this line.
- **LLM tokens** scale with conversational volume. Claude is $5/$25 per 1M in/out (Opus 4.8), $3/$15 (Sonnet 4.6), $1/$5 (Haiku 4.5), with **prompt caching cutting cached input 90% and batch 50%** [Claude Platform Docs / platform.claude.com/docs/en/about-claude/pricing]. Because our parse path is *schema-forced and short*, most calls are cheap — but a chatty "talk to your money" UX can run hot. Route deterministic/cheap paths to Haiku, reserve Opus for genuine planning, cache the system prompt.
- **Audits** are one-time capex, not run-rate, and they are large. Top-tier firms (Trail of Bits, OpenZeppelin) bill ~**$25k per engineer-week**; a serious wallet review is typically 3–8 engineer-weeks → **$50k–200k**, with the market-wide range $5k–250k+ [7BlockLabs / 7blocklabs.com/blog]. Note the honest nuance: a pure-EOA non-custodial wallet has *no Solidity to audit* — the audit target is the **on-device crypto core, the deterministic gate, and the client build supply chain** (a Cure53/Least Authority-style appsec+crypto review), which is priced similarly to a contract audit. If we later add account-abstraction/session-key contracts, that's a *second, separate* audit.

Phased adoption roadmap

Phase 0 — MVP / launch (testnet + guarded-capped mainnet, dogfooding). Everything runs on free tiers. Alchemy free (30M CU) and Helius free (1M credits, devnet) cover RPC; Claude usage is a few dollars; SIWE session auth is self-hosted (already built); Reown/WalletConnect is free to integrate. Infra: a single managed Postgres + Redis on free/hobby tiers, one container host. Observability: Sentry free (5k errors). **No audit yet** — internal adversarial review only (the repo already runs its own threat models and known-answer conformance tests). *Run-rate: ~$0–150/month.* Deferred: address screening, tx-scanning threat intel, multi-region, dedicated nodes, embedded wallets.

Phase 1 — private beta (real users, real mainnet, capped funds). This is where the first real bills appear. Helius jumps to **Business $499/month** the moment you touch Solana mainnet; Alchemy PAYG lands somewhere in **$50–400/month** depending on polling; Claude scales to **$100–1,000/month** with caching. Add Supabase/Neon Pro (~$25–50 + compute), Upstash Redis ($10–50), a real container tier ($25–85), Cloudflare Pro/Business ($20–200) for WAF + rate-limit depth, and Sentry paid ($26–100). Now integrate **transaction-scanning threat intel (Blockaid)** and, if any fiat/compliance surface exists, **address screening (TRM/Chainalysis)** — both enterprise-quoted. **Commission the pre-GA audit here** ($50k–150k one-time). *Run-rate: ~$1k–3k/month + audit capex.*

Phase 2 — public / scale. RPC becomes the dominant line: Alchemy scale traffic $500–5k+/month, Helius Business/Professional $499–999 + autoscaling overages (dedicated nodes $2,900+ if latency-critical). LLM $2k–20k+/month at real MAU (aggressively cached, Haiku-tiered, batched where async). Infra goes HA/multi-region: Upstash Prod Pack (+$200/db for SLA + SOC-2 + encryption-at-rest), scaled Postgres, container fleet $500–5k, Cloudflare Business/Enterprise ($200→custom). Observability $1–3k (Sentry) and optionally Datadog for infra ($5–15k at 50 hosts). Security becomes continuous: recurring audits + a **bug-bounty pool (Immunefi/Cantina)** as capex, plus enterprise Blockaid + screening contracts. *Run-rate: ~$10k–50k+/month + recurring audit/bounty capex.*

Rough monthly cost tiers (order-of-magnitude, honest)

DRIVER	PHASE 0 (MVP)	PHASE 1 (BETA)	PHASE 2 (SCALE)
EVM RPC (Alchemy PAYG)	$0 (free 30M CU)	$50–400	$500–5,000+
Solana RPC (Helius)	$0 (devnet)	**$499 (mainnet floor)**	$499–2,900+
LLM (Claude, cached)	$1–20	$100–1,000	$2,000–20,000+

DRIVER	PHASE 0 (MVP)	PHASE 1 (BETA)	PHASE 2 (SCALE)
Managed Postgres + Redis	$0–35	$50–150	$500–3,000
Container hosting	$0–25	$25–150	$500–5,000
CDN / WAF (Cloudflare)	$0	$20–200	$200–custom
Observability (Sentry ± Datadog)	$0	$26–200	$1,000–15,000
Threat intel + screening	deferred	enterprise (quote)	enterprise (quote)
Software run-rate	**~$0–150/mo**	**~$1k–3k/mo**	**~$10k–50k+/mo**
Audits (one-time capex)	internal only	**$50k–150k**	recurring + bounty pool

Buy vs build — the doctrine draws the line for you

The rule is mechanical: **buy anything a breach of which can only cost privacy or uptime; build in-house anything that sits on the fund-loss trust boundary.** Vendors live in Zones 1–4; the deterministic cores live in Zone 0 and are the product's moat.

- **Buy (commodity, non-custodial by construction):** RPC (Alchemy, Helius), LLM (Anthropic Claude), managed Postgres/Redis, observability (Sentry), CDN/WAF (Cloudflare), wallet-connect UX (Reown), price feeds (CoinGecko), tx-scanning threat intel (Blockaid), and address screening (TRM) *if/when a compliance surface exists*. None of these ever needs a private key or seed; the worst case is a leaked address/balance/intent — a privacy incident, not fund loss.
- **Build & keep in-house (never outsource — these *are* the security guarantee):** the on-device keystore/vault (scrypt + AES-256-GCM), HD derivation and multi-chain signing, the **deterministic risk/policy/capability gate** that can only refuse, the schema-forced intent parser boundary, SIWE session issuance + JWT revocation, the router scoring, and the settlement state machine. These are pure, exhaustively-tested cores; buying them would mean importing someone else's trust boundary — a doctrine violation.
- **The one "buy" that violates doctrine — flag it explicitly:** embedded / MPC-wallet infrastructure (Privy, Turnkey, Fireblocks-owned Dynamic post its Oct 2025 acquisition). Privy free to 50k signatures / $1M volume; Dynamic uses TSS-MPC that "never constructs a full private key" [Fireblocks / fireblocks.com/report, Dynamic / dynamic.xyz/features/wallet-infrastructure]. These are excellent *onboarding* rails but each reintroduces a **custody question**: a server-side key share, an SSS reconstruction, or a hosted enclave means a secret exists off the user's device. For Intent Wallet's north star this is **disqualified as the default.** If email/social onboarding is ever wanted, it must be an *explicitly labeled, opt-in "assisted" lane* clearly distinguished from the true self-custody path — never the silent default, and never the thing that holds funds for our promise-keeping users.

Recommended reference stack

LAYER	CHOSEN SERVICE(S)	WHY
EVM RPC	**Alchemy** (already wired)	Transparent CU pay-as-you-go, generous free tier, deep multichain L2 coverage; read/relay only — never holds a key.
Solana RPC	**Helius** (already wired)	Best-in-class Solana data/websockets; budget for the $499 mainnet floor. Read/relay only.
LLM (intent parse)	**Anthropic Claude** — Haiku for cheap paths, Sonnet/Opus for planning	Schema-forced boundary; caching + batch make it cheap; AI has zero signing authority by design.
Auth / identity	**Self-hosted SIWE + JWT** (built), Reown for wallet connect	Server learns a public principal, issues a session, never a secret. Doctrine-native.
Wallet core / signing	**In-house** `@intent-wallet/core`	The trust boundary; keys generated/used on-device, never bought or outsourced.
Safety gate	**In-house risk/policy/capabilities** + **Blockaid** for external threat intel	Deterministic code disposes; Blockaid *informs* the verdict, never overrides it. Fail closed.
Managed data	**Neon or Supabase (Postgres) + Upstash (Redis)**	Cheap to start, usage-based, SOC-2 available (Upstash Prod Pack); holds only privacy data, never secrets.
Hosting	**Railway/Render (beta) → AWS/GCP (scale)**	Start on DX-first PaaS; graduate to own-cloud (Fargate/Cloud Run) when RPC/LLM egress dominates.
Edge / WAF	**Cloudflare** (Pro→Business)	Unmetered DDoS on every tier, WAF + rate-limit depth at Business ($200); protects availability.
Observability	**Sentry** (add Datadog only at scale)	Errors + tracing cover 90% of needs cheaply; secrets never logged (doctrine), so scrubbing config is mandatory.

LAYER	CHOSEN SERVICE(S)	WHY
Price feed	**CoinGecko API**	$129 Analyst gives real-time + websockets at ~7x under CoinMarketCap Pro; a read-only oracle, treated as Zone-4 (validated, bounded).
Screening (conditional)	**TRM / Chainalysis** — only if a fiat/compliance surface appears	Enterprise-priced; defer until a jurisdiction actually requires it.

Recommendation for Intent Wallet. *MVP pick:* ship on the stack that's already wired — **Alchemy + Helius + Claude + self-hosted SIWE**, everything else on free/hobby tiers, **no embedded wallet, no audit yet, internal adversarial review only.** Run-rate near zero; the only rule is polling discipline so RPC never surprises you. *Scale pick:* the same spine, hardened — **Helius Business, cached/tiered Claude, HA Postgres+Redis, Cloudflare Business, Sentry, Blockaid in the gate, TRM only if compliance demands it** — and, critically, **budget the $50k–150k pre-GA audit of the on-device core as the single most important line item on the roadmap.** Buy the convenience layers freely; build and guard the cores that touch funds; keep embedded/MPC custody off the default path forever. That is how the bill stays honest and the promise stays true.

Sources: Alchemy Pricing · Helius Pricing · Claude Platform Docs — Pricing · 7BlockLabs — Audit Costs · Supabase Pricing · Upstash Pricing · Sentry vs Datadog — Better Stack · Cloudflare Plans · CoinGecko API Pricing · Railway/Render/Fly — Northflank · Privy Pricing · Fireblocks — Embedded Wallet Comparison · Reown FAQ